Product Manager

Beginner handbook

Laravel MVC, product CRUD, Breeze authentication, database, Blade, CSS, JavaScript, and developer tools.

Plain-language terms and concepts, study guide, complete file map, syntax/API reference, eight Lab 3 answers, and practice exercises.

Prepared October 13, 2026. Read alongside the application source; examples explain the code and do not introduce unimplemented features.

Start with the request journey, then follow one operation through its route, checks, database call, and response. Use the file map and reference tables when a name or expression is unfamiliar.

Contents

Product Manager learning pack 3

Terms and concepts before reading the code 5

Understanding Product Manager from request to response 20

Laboratory 3 guide-question answers 34

Practice, checks, and demonstration 37

Syntax and API reference used by this application 42

Application file map 50

Verification of the documented application 66

Instructor questions: files, lines, and code to show 68

Product Manager learning pack

Start with this page. The application is in `app-crud/`; paths in the guides are relative to that folder unless a command shows otherwise.

Reading order

1. Terms and concepts: plain definitions, reasons, project examples, and short checks before reading unfamiliar code.
2. Complete beginner guide: web requests, PHP syntax, MVC, CRUD, authentication, database, Blade, CSS, JavaScript, and tools.
3. File map: every application source file and how it connects.
4. Lab 3 answers: all eight PDF guide questions, with examples and the code to point to during a demonstration.
5. Practice and demonstration: exercises, answer checks, troubleshooting, and the required submission evidence.
6. Syntax and API reference: lookup tables for the operators, helpers, model/request methods, Blade directives, and browser APIs.
7. Instructor code locator: exact file names, current lines, excerpts, and spoken explanations for each PDF question.

Read one chapter, open its files, and trace one example. Close the guide before explaining the example aloud. Revisit anything you could only repeat from memory.

Which files should be edited?

The Product controller/model/routes, custom views, and public CSS/JavaScript are the main learning files. Generated Breeze controllers and request classes are also part of the application and are commented. Configuration/build files have short connection notes; their values are explained in the file map.

Leave `vendor/`, `node_modules/`, runtime caches, compiled builds, and lockfile contents to their tools. JSON does not allow comments, so package/lock definitions are explained in the guide rather than made invalid by inserting `//`. Keep the real `.env` private. Documentation uses placeholders, never actual mail passwords, application keys, session cookies, or reset tokens.

What currently happens?

Action	Current result
Open `/` as a guest	Redirect to Login
Open `/` while signed in	Redirect to Products
Successful login	Intended protected page, otherwise Products
Logout	Session cleared; redirect to Login
Successful registration	Automatically signed in; Dashboard
Successful password reset	Login with reset-success status

Action	Current result
Guest requests a product route	Login required
Signed-in user requests products	Shared product data is accessible

Registration and login have different default redirects in the current code. Email verification routes are generated, but verification is not enforced by the User model. There is no administrator role or per-user product ownership policy.

Version history and documentation

`lab2-complete` is the GitHub checkpoint before Breeze installation. The older `LAB2_STUDY_GUIDE.md` and PDF describe that stage. This learning pack describes the current combined Lab 2/3 application. The comments and guides are local changes until a later reviewed Git commit/push; nothing is silently published by writing them.

Humanizer is installed in the Windows user's global `.agents/skills/humanizer` folder. It was used to keep these explanations plain and precise. New Codex turns can discover it; installing a skill does not run it continuously in the background.

Terms and concepts before reading the code

Read the explanations of requests, methods, routes, and middleware first. The later sections define the other terms you meet in this project. Each example uses Product Manager or is clearly labelled as a learning example.

1. Browser, server, and request

The browser is the program you use to view the website. The server is the program that receives the browser's requests and returns a response. They can run on the same laptop and still have separate responsibilities. PHP runs in the server; HTML, CSS, and browser JavaScript are used by the browser.

A request is a message asking the server to do something. It contains a method, an address/path, headers, and sometimes submitted data. A response is the server's reply, containing a status, headers, and often a body such as HTML.

When you open `/product`, the browser asks for the product list. When you press Save Product, it submits the entered values. Those are different requests even though the list and save route both use `/product`.

Term	Plain meaning	Example / reason in this app
Client	Program requesting a service	The browser is the web client
Browser	Program rendering pages and running browser JavaScript	Displays the table; Show/Hide runs here
Server	Program handling requests and providing responses	artisan serve runs the local PHP web server
Frontend	Interface and interactions the user sees	Labels, buttons, CSS, theme controls
Backend	Server-side processing and services	Controllers, validation, authentication, database queries
Request	Message sent to ask for a resource/action	POST with product name/quantity/price
Response	Reply to a request	HTML list or a redirect to Login
HTTP	The web's request/response protocol	Method and URL identify the intended request
Protocol	Rules for communication	HTTP handles pages; SMTP handles outgoing mail
HTTPS	HTTP protected by TLS in transit	Needed for secure public deployment; local lab uses HTTP
URL	Address of a resource	http://127.0.0.1:8000/product
Host	Machine/service address part of a URL	127.0.0.1 is this computer
localhost / loopback	Addresses for the machine itself	Local browser contacts local server
Port	Number identifying a service endpoint	8000 web server; 3307 database
Path	Part selecting a resource/action	/login, /product/create
Query string	Extra URL values after question mark	The reset link supplies an email query value

Term	Plain meaning	Example / reason in this app
Header	Request/response metadata	Content-Type describes how to interpret the body
Body	Main message content	Form fields in a POST; HTML in a response
Payload	Data carried in a message/storage field	Session payload or submitted form values
Endpoint	Address and method handled by the application	POST /product reaches store()
Redirect	Response instructing the browser to visit another address	After save, visit GET /product
Status code	Number describing response outcome	200 succeeds; 302 redirects; 404 is not found

An endpoint isn't just a visible button. A caller can send a request without clicking your page, so server rules still matter when the UI looks correct.

2. GET, POST, PUT, PATCH, and DELETE

An HTTP method, also called a verb, tells the server the purpose of a request. GET retrieves a representation, POST submits data, PUT replaces a representation, PATCH applies a partial change, and DELETE requests removal. HEAD returns GET-like response headers without its body. OPTIONS reports communication options. These meanings come from the HTTP method definitions; a program must implement them sensibly. HTTP methods reference

Here is how our actual routes use them:

Method and path	Meaning here	Why this choice is used
GET /product	Display saved products	Viewing should not insert/delete records
GET /product/create	Display a blank form	Opening the form is not saving it
POST /product	Submit a new product	The request asks the server to create a record
GET /product/{product}/edit	Display the selected record's form	Reading values before an update
PUT /product/{product}	Submit the complete editable product fields	Our form sends the values for that record
DELETE /product/{product}	Remove the selected record	The request explicitly changes stored data
POST /login	Submit account credentials	Process a sign-in attempt, not merely show the form
POST /logout	End the session authentication	Logout changes state and needs CSRF protection
PATCH /profile	Submit profile information changes	The generated route uses partial-update semantics

GET does not mean "run an SQL SELECT" in every application. It is an HTTP request method. In our list route the controller happens to perform a database read. POST does not mean "insert one row" every time either: login, logout, and reset requests also use it. The route decides which controller processes it.

```
Route::get('/product', [ProductController::class, 'index']);
Route::post('/product', [ProductController::class, 'store']);
```

Read the first line as: a GET request to this path should call index. Read the second as: a POST to that same path should call store. Method plus path chooses the route. The uppercase methods describe HTTP; Laravel's lowercase get/post are PHP method names registering those routes.

HTML forms and method spoofing

Ordinary HTML forms support GET/POST submission. Our update form uses POST and `@method('PUT')`, which generates a hidden `_method` field Laravel interprets. "Spoofing" here means representing the intended method through that hidden field, not bypassing authentication. Delete uses the same mechanism for DELETE.

```
<form method="POST" action="{{ route('product.update', $product) }}">
    @csrf
    @method('PUT')
</form>
```

Keep the named route, CSRF token, and method field connected. Changing one word without understanding the receiving route can produce a Method Not Allowed error.

3. A route and middleware are different jobs

A route associates a method/path with code. Middleware surrounds request handling with checks or supporting behavior. It can allow the request to continue, redirect it, or reject it before the controller. Some middleware also processes the response. For this application, auth checks login; web middleware supports sessions and CSRF validation. Laravel middleware reference

The practical order for a protected request is:

```
Request -> matched route and applicable middleware -> controller
               |
               +-> guest? auth redirects to Login
```

You can imagine a check at a doorway, but the real implementation runs on each request. It is not a person checking once when the browser opens. Hiding a link does not prevent someone requesting that endpoint directly.

```
Route::middleware('auth')->group(function () {
    Route::get('/product', [ProductController::class, 'index']);
});
```

`middleware('auth')` selects a named check. `group(...)` gives it to the routes registered inside the callback. The closure's braces group PHP statements; the middleware is not the controller's own validation rules.

Term	Plain meaning	Example / reason in this app
Route	Map from request method/path to handling code	web.php connects /product to index
Named route	Reusable label for generating URLs	product.index is a label, not a file path
Route parameter	Variable segment of a path	{product} takes the selected product ID
Route model binding	Laravel loads the model matching the parameter	Product $product receives a saved row object
Middleware	Request/response processing around the handler	auth stops guest access before CRUD runs
Middleware group	A reusable collection of middleware	web provides sessions/cookies/CSRF behavior

Term	Plain meaning	Example / reason in this app
Route group	Routes sharing selected attributes/checks	Six product routes share auth
Alias	Short name mapped to middleware	auth and guest select Laravel's checks
Guest	Visitor without an authenticated session	Can open Login but is redirected from Products
Authenticated user	User whose session identifies a signed-in account	May enter the current shared product routes
Intended destination	Protected address requested before login	Visiting Add Product as guest returns there after login
Fallback	Result used when the preferred value/destination is absent	Products is login's default destination

Middleware compared with validation

auth asks whether the session is signed in. Validation asks whether submitted values satisfy rules. A guest should be stopped before attempting a write even if every submitted value is valid. A signed-in user still gets validation errors for an invalid price. These checks solve separate problems.

4. MVC, framework, and application structure

MVC means Model, View, Controller. A controller coordinates one action; a model works with stored records; a view presents values and form controls. Laravel is a framework providing structure and services so these parts can work together.

Term	Plain meaning	Example / reason in this app
Application	Program solving a user's task	Product Manager combines CRUD and accounts
Framework	Reusable structure/services for building applications	Laravel supplies routing, sessions, validation
Library/package	Reusable code installed as a dependency	Symfony Mailer is used by Laravel's mail services
Dependency	Code/tool required by other code	Composer installs Laravel's dependencies
MVC	Responsibilities split into model/view/controller	Product, products.index, ProductController
Model	Class representing/interacting with records	Product connects by convention to products
View	Template producing the displayed interface	login.blade.php produces a login form
Controller	Request-handling class coordinating actions	store validates, saves, and returns a redirect
Request class	Object describing the incoming request, optionally with rules	LoginRequest validates and authenticates login input

Term	Plain meaning	Example / reason in this app
FormRequest	Laravel request class with validation hooks	ProfileUpdateRequest provides profile rules
Service	Object/functionality performing a focused application job	Password broker handles token verification
Facade	Laravel class-style access to a service	Auth::attempt uses the authentication service
Helper	Convenient function offered by PHP/Laravel	view(), route(), compact()
Service container	Laravel's mechanism for constructing/resolving dependencies	Supplies LoginRequest to the controller method
Dependency injection	Receive a needed object as an argument	Request $request is provided rather than manually built
Provider	Startup class registering/booting application services	AppServiceProvider is listed in bootstrap/providers.php
Bootstrap	Startup work preparing an application	bootstrap/app.php registers routes and configuration
Entry point	First application file processing that kind of input	public/index.php for HTTP; artisan for commands
Autoloader	Finds class files when referenced	Composer's vendor/autoload.php
Scaffold	Generated starting code meant to inspect/customize	Breeze's controllers, views, tests
Convention	Expected naming/structure that avoids extra configuration	Product maps to products unless configured otherwise
Configuration	Values choosing how services operate	Database connection, mail transport, session driver
Environment	Local/deployment settings outside source logic	.env provides host/port/private credentials

Why separate layers? To change how a button looks, edit a view/style without changing database queries. To adjust accepted prices, edit server validation without drawing a new page. The file map tells you which responsibility belongs to the file you are opening.

5. Database and CRUD language

A database stores organized data. A table is one record type, a row is one record, and a column is one attribute. A model is PHP code that interacts with this storage; it is not the table itself.

Term	Plain meaning	Example / reason in this app
CRUD	Create, Read, Update, Delete operations	Four ways to work with products
Database	Persistent organized data store	paraguya_crud_lab2 on MariaDB
Table	Records with a shared structure	products or users

Term	Plain meaning	Example / reason in this app
Row/record	One stored item	One Apple product
Column/field	Named attribute within that structure	name, qty, price
Schema	Definition of table/column structure	Products migration describes it
Migration	Versioned program for schema changes	up creates; down reverses
SQL	Language for database operations	SELECT reads rows
Query	Instruction asking the database for work/data	Product::all produces a read query
ORM	Maps database data to program objects	Eloquent represents rows as Product/User objects
Eloquent	Laravel's ORM implementation	create/update/delete methods used here
Primary key	Identifier uniquely distinguishing a row	products.id
Auto-increment	Database assigns the next numeric identifier	New product gets an ID automatically
Unique constraint	Database requires a value to be unique	users.email cannot duplicate another row
Index	Data structure helping database lookups	Sessions activity/email lookup indexes
Foreign key	Database-enforced relationship to another key	A possible products.user_id constraint is not implemented here
Relationship	Defined connection between records/models	Product ownership would need extra model/schema work
Data type	Permitted representation of a value/column	Integer qty; decimal price; text description
Decimal precision/scale	Total digits / digits after decimal point	decimal(8,2) allows six before, two after
NULL/nullable	Absence of a value / allowed absence	Optional description may be NULL
Timestamp	Stored date/time value	created_at and updated_at
Persistence	Data remains after the request/program state ends	A saved product survives a refresh
Collection	Object holding multiple values/models	products is the collection used by the list
Mass assignment	Set multiple model attributes from an array	Product::create($data)
Fillable	List of allowed mass-assignment keys	name/qty/price/description
Cast	Conversion for a model attribute	User date or hashed password assignment
Serialization	Convert an object into a transport/storage representation	User converted to array/JSON omits hidden fields

Term	Plain meaning	Example / reason in this app
Seed	Insert optional initial/sample records	DatabaseSeeder, not normal user registration
Factory	Program generating record fixtures	UserFactory creates test users
Fixture	Known sample data used in a test	Disposable Test Apple record
Rollback	Reverse a recorded migration	Dropping the products table also removes its rows
Backup	Separate recoverable copy of data/source	SQL export before Breeze changes

Why use validation if a column has a data type? A numeric column can still accept values your application does not want. Quantity should not be negative here, so the controller has min:0. A fixed decimal column may round input in ways you do not intend, so the controller rejects three-decimal prices before saving.

6. Accounts, security, and email terms

Authentication asks which user/session is signed in. Authorization asks whether that user may do a particular action. Our auth group implements a login boundary; it does not implement administrators or product ownership.

Term	Plain meaning	Example / reason in this app
Registration	Create a saved account	Register inserts a User
Authentication	Verify credentials/establish signed-in identity	Login checks the password hash
Authorization	Decide permissions for that user/action	Owner/admin-only deletion would need additional rules
Credential	Value used to prove/access an identity/service	Login password or SMTP credential
Session	Server-managed state connecting a user's requests	Session identifies the signed-in account
Cookie	Small value browser stores/sends for a site	Session cookie connects requests to session data
Guard	Laravel mechanism checking authentication	web guard uses browser sessions
User provider	How Laravel retrieves account records	Eloquent provider loads User
Remember token	Value supporting remembered authentication	users.remember_token; not readable password
CSRF	Cross-site request forgery, a forged action using a victim's session	Laravel checks form token before accepting writes
CSRF token	Session-related value required by protected forms	@csrf creates hidden _token

Term	Plain meaning	Example / reason in this app
Hash	One-way password representation for checking	Hash::make before storage
Encryption	Reversible protection using a key	Laravel encryption uses APP_KEY; password hashes are different
Session fixation	Attacker tries to reuse a known session identity	Regenerate session ID after login
Rate limiting/throttling	Restrict repeated actions over time	Login failures and reset-email cooldown
Validation	Check input against acceptance rules	required, email, confirmed, decimal
Sanitization	Transform/clean input for a particular purpose	Different concept from checking a value against rules
Escaping	Encode text for safe display in a context	Blade double braces escape HTML
XSS	Cross-site scripting, unwanted script in page content	Escaped user product names help prevent HTML/script insertion
SQL injection	Input changes SQL instructions instead of being data	Bound values in these Eloquent calls prevent interpolation mistakes
Parameter binding	Send query values separately from SQL structure	The product create/update values are bound
Role	Named category of user permissions	Admin would be a new role feature, not supplied by auth alone
Policy/gate	Laravel authorization decision code	Possible owner-only update extension
Password broker	Laravel service managing reset requests/tokens	sendResetLink and reset calls
Reset token	Temporary proof required to change a password	Token in the newest emailed reset URL
Expiry	Time after which a value/request stops being valid	Reset expiry is 60 minutes here
Email verification	Prove control of an account email	Requires extra model configuration in this app
SMTP	Protocol used to submit outgoing email	Mailtrap/Gmail transport
Mailer	Component selecting/building/sending email	Laravel mailer uses smtp here
Transport	Mechanism carrying a message to delivery	Symfony SMTP transport
TLS	Protection for data while transmitted	SMTP can upgrade through STARTTLS
STARTTLS	Upgrade an existing connection to TLS	Used with our smtp schemes/provider ports

Term	Plain meaning	Example / reason in this app
App Password	Provider-generated credential for an application	Gmail sender credential, separate from Laravel login
Sandbox	Isolated test environment	Mailtrap captures email in a test inbox
Notification	Application message sent through a channel	ResetPassword notification uses email
Event/listener	Occurrence and code responding to it	Registered event allows account-created handling
Exception	Failure interrupting normal execution	ValidationException returns field errors through Laravel
Error bag	Named collection of form errors	Profile password/delete bags stay separate
Flash message	Session feedback intended for the next request	Product saved successfully after redirect

A password input's dots only change its display. Show/Hide does not alter hashing. Similarly, a hidden token input is still inspectable and editable by the browser user. The server verifies it instead of trusting it just because it is hidden.

7. Programming-language terms

Term	Plain meaning	Example / reason in this app
Syntax	Rules for writing valid code	PHP variable prefix, braces, semicolons
Statement	One executable instruction	Assign a value or return a response
Expression	Code producing a value	auth()->check() produces a boolean
Variable	Named storage/reference for a value	$products
Literal	Value written directly in code	60, 'Apple', true
String	Text value	Product name
Integer	Whole number	Quantity
Boolean	true/false value	Remember me selected or not
Array	Collection indexed by keys	Validated field values
Key/value	Array identifier / its associated data	'qty' maps to 12
Function	Callable unit of work	route() helper
Method	Function belonging to a class/object	Controller index() or model update()
Parameter	Variable declared to receive a call's input	Request $request
Argument	Actual value passed to a call	'product.index' passed to route()

Term	Plain meaning	Example / reason in this app
Return value	Result sent back to the caller	View or redirect
Class	Definition of object data/behavior	ProductController or User
Object/instance	A concrete class value	Selected Product record
Property/attribute	Data associated with an object	$product->name
Inheritance	Reuse/extend a parent class	Product extends Model
Visibility	Which code may access a class member	public method; protected fillable
Namespace	Organized class-name scope	App\Models
Import	Bring a class/module name into this file	use Product or JS import Alpine
Interface	Contract a class declares it meets	MustVerifyEmail controls enforcement eligibility
Trait	Shared class behavior included in a class	Notifiable
Closure	Anonymous function, sometimes capturing outer values	Route group callback
Callback	Function supplied to be called at a later processing step	Reset broker invokes password-change callback
Operator	Symbol performing a language operation	= assigns; === compares strictly
Conditional/branch	Select execution based on a condition	Home guest vs signed-in redirect
Loop/iteration	Repeat work for each item	@forelse product rows
Scope	Where a variable/name is accessible	Closure captures Request with use
Chain	Calls using the previous call's returned object	redirect()->route()->with()
Constant	Named fixed value	Password::PASSWORD_RESET status
Type hint	Declared expected parameter/return type	LoginRequest and RedirectResponse
Null	No value present	Optional description
Fallback/default	Value used when none was supplied/found	old(field, savedValue)
Comment	Explanation ignored by execution	// describes a route connection

One method name can appear in unrelated classes. Controller `update()` handles an HTTP action; model `update()` saves a record. Identify the object/class before assuming two calls do the same work.

8. HTML, Blade, CSS, and browser interaction terms

Term	Plain meaning	Example / reason in this app
HTML	Markup describing document structure	Forms, headings, table
Element/tag	One markup object / its written delimiter	input element or opening form tag
Attribute	Additional element setting	name, type, class, required
Form	Controls submitted together to a server action	Registration fields
Label	Field name linked to its input	for=email matches id=email
Submit	Send the form values	Save Product POST
Hidden input	Submitted control not drawn visibly	Reset token and _method
Placeholder	Hint when a text field is empty	you@example.com; not a field label
Autocomplete	Hint to browser/password managers	new-password vs current-password
Blade	Laravel server template language	Directives and escaped expressions
Template	Source for producing repeated page structure	products.index
Layout	Shared page shell	Navbar and content area
Section/yield	Provided/included inheritance-layout content	Product content section
Component	Reusable view/class with props/content	x-guest-layout
Slot	Content inserted into a component	Login form becomes $slot
Prop	Input/default to a component	Dropdown alignment
Partial/include	Smaller view inserted in another	Profile forms
Directive	Blade template instruction	@csrf, @if, @extends
Rendering	Convert template/data into displayed output	Blade becomes HTML
DOM	Browser's structured representation of the page	JS finds and updates input elements
Selector	Pattern finding/styling matching elements	.auth-input input[type=password]
CSS	Rules controlling appearance/layout	Glass panel background and spacing
Class	Markup token selected by CSS/JS	auth-field

Term	Plain meaning	Example / reason in this app
Cascade/specificity	Rules deciding which CSS declaration wins	Password padding must beat generic input padding
Box model	Content, padding, border, margin sizing	border-box includes padding in declared width
Padding/margin	Internal/external spacing	Space within/between panels
Flexbox	Layout method arranging children	Header/action alignment
Responsive design	Layout adapts to available screen space	Mobile form padding and table scrolling
Viewport	Browser's available page display area	600px media query threshold
Media query	Conditional CSS rules	Smaller screens or reduced motion
Transparency/alpha	How much backdrop shows through	rgba glass fill
Gradient	Gradual color/light variation	Neutral background and button sheen
Backdrop blur	Blur pixels behind an element	Frosted navbar
Pseudo-class/element	State selector / generated decorative part	:hover and navbar ::after divider
Event/listener	Browser action and callback responding	Theme button click
Timer	Scheduled future browser callback	Five-second success-banner dismissal
LocalStorage	Browser site storage	Theme choice, not login password
SVG	Vector image markup	Envelope and lock icons
Focus	Element receiving keyboard input	Tab to checkbox or Show button
Accessibility	Usability for different input/assistive needs	Labels, focus rings, readable error text
ARIA	Extra accessibility semantics	pressed state on theme/password toggle
Utility class	Single-purpose styling class	Generated Tailwind flex/padding classes

9. Tools, builds, and version-control terms

Term	Plain meaning	Example / reason in this app
Terminal/CLI	Text interface for commands	PowerShell runs Artisan
Working directory	Folder a command runs from	app-crud contains artisan
Artisan	Laravel command-line entry/tool	serve, migrate, route:list, test
Composer	PHP dependency manager	Installs Breeze and Laravel packages

Term	Plain meaning	Example / reason in this app
npm	Node package manager/script runner	Installs/builds frontend dependencies
Node.js	JavaScript runtime outside the browser	Runs Vite build tooling
Vite	Frontend build/development tool	Produces compiled CSS/JS
Build	Transform source into served artifacts	npm run build
Manifest	File describing produced build artifacts	Vite manifest maps entry assets
Lockfile	Exact resolved dependency versions	composer.lock / package-lock.json
Cache	Stored reusable work/data	Compiled Blade or config; not source to edit
Clear cache	Remove that stored work so it is rebuilt/read again	view:clear / config:clear
Runtime artifact	File produced while the app/tools run	Compiled views and logs
Log	Recorded events/errors	storage/logs; avoid exposing secrets
Git	Source-history tool	Tracks the project versions
Repository	Files and version history under Git	The public coursework repository
Commit	Saved snapshot in history	Lab 2 checkpoint commit
Branch	Named line of commits	main
Tag	Named reference to a checkpoint	lab2-complete
Remote	Repository location elsewhere	origin on GitHub
Push	Send local commits/references to a remote	Publish a reviewed Lab 3 checkpoint
Diff	Comparison of versions	Review changed routes/comments
Ignore rule	Pattern excluding local files from tracking	.env and dependencies
Test/assertion	Executed check / expected result it verifies	assertRedirect verifies login destination
Unit/feature test	Small behavior / app request-flow test	Sample assertion vs authentication request
Fake/mock	Controlled replacement during tests	Notification::fake avoids real email
Test isolation	Checks separated from real data/services	SQLite :memory: and array mail

Git tracks code and migrations. It does not automatically back up MySQL rows. Clearing a view cache does not mean deleting tables. Knowing which kind of stored thing you are changing prevents destructive "cleanup" guesses.

10. Connect the terms to one action

When Save Product is clicked, the HTML form submits a POST request containing field values. Laravel matches the route. web middleware checks the session/CSRF and auth requires a signed-in user. The controller validates values, then calls the Eloquent model to insert a database row. It returns a redirect with a flash message. The browser requests the list; Blade renders it and CSS styles it. JavaScript later removes the banner after five seconds.

That explanation connects the terms to a sequence. A standalone definition becomes easier to remember when you can point to its role in this action.

11. Short checks to strengthen understanding

Try each prediction before reading its answer.

GET versus POST

Opening /product/create does what? Pressing Save Product does what?

Answer: GET displays a form. POST submits its values to store, which validates and inserts. Opening the form should not insert a row.

Route versus middleware

If a guest sends a valid POST directly to /product, which check stops the request?

Answer: auth middleware stops guest access before store runs. Valid input cannot replace the required authenticated session.

Middleware versus validation

A logged-in user submits price 1.999. Why is it rejected?

Answer: login passes auth, but controller decimal validation rejects the value. Different checks evaluate identity and submitted data.

Class versus object

What is Product, and what is $product after route model binding?

Answer: Product is the model class. $product is the loaded object representing the selected record. Its ->name reads that object's attribute.

Cookie versus localStorage

Does changing the saved theme preference change authentication?

Answer: no. Our localStorage value controls appearance. Laravel authentication uses its session/cookie mechanism.

Hash versus password visibility

Does pressing Show reveal a password decrypted from the users table?

Answer: no. It displays the currently typed field value. The database stores a hash; the Show button does not decrypt or read it.

Migration versus model

Which creates the products columns, and which saves a new Apple row?

Answer: the migration creates schema; Product::create saves the row using it.

Redirect versus view

Does redirect()->route('product.index') render the table by itself?

Answer: no. The browser follows a new request to the list route, whose controller returns the rendered view.

Backend versus frontend

Does changing CSS remove a product from MySQL?

Answer: no. CSS changes presentation; the server-side delete action changes data.

Authentication versus authorization

Does being logged in prove a user is an administrator?

Answer: no. It establishes authentication. Admin/ownership permissions need additional data and authorization checks.

12. Learn to explain rather than repeat

For a term, write four short lines: its meaning, the problem it solves, the file where it appears, and one prediction about removing/changing it. Example:

```
auth middleware: requires a signed-in session before protected code runs.
Problem: guests could otherwise call product endpoints directly.
Location: six routes inside the auth group in routes/web.php.
Prediction: removing it would remove that login check from those endpoints.
```

Then check the source and correct the explanation. In a safe test environment, write an assertion for the predicted result instead of disabling protection on the real app. Explain it again the next day without looking. Add a concrete example when two terms seem similar. Use the complete guide for code walkthroughs and the API reference when the unfamiliar thing is an expression/method name.

Understanding Product Manager from request to response

This is a guide to the code in this project, including the language features it uses. Examples marked as exercises are illustrations, not code already added to the application. Follow the chapters in order on your first pass.

If terms such as middleware, POST, session, or model are unfamiliar, start with Terms and concepts. It explains what they mean, why the application uses them, and the code/file where each one appears.

1. The separate programs involved

Your browser displays HTML, uses CSS for appearance, and executes JavaScript for interactions. It does not execute PHP. PHP runs on the server, where Laravel decides which code handles a request and how data is loaded or saved.

XAMPP supplies PHP and a MySQL-compatible MariaDB database. Composer installs PHP packages. Node.js/npm run frontend tools such as Vite; they are not the database server. VS Code edits files. Git saves source history. phpMyAdmin is a web interface for examining the database. Each program has a different job.

Technology	Job in this project	Example
PHP	Server-side programming	ProductController validates and saves input
Laravel	Web framework and services	Routing, sessions, Eloquent, validation
Blade	Server-side HTML templates	Loop through products and display errors
HTML	Page structure and form semantics	A labelled input and submit button
CSS	Appearance and responsive layout	Glass panels and form spacing
JavaScript	Browser interaction	Show/Hide password and theme switch
SQL database	Persistent tables and records	Products remain after browser refresh
Breeze	Generated authentication scaffold	Login, registration, recovery, profile
Tailwind/Alpine	Generated Breeze UI tools	Dashboard utilities and account menus
Vite	Frontend asset build	Compiles resources CSS/JS into public/build

The customized authentication/product pages use Inter. Dashboard/Profile retain generated Breeze styling. Selecting Inter in plain CSS does not automatically rewrite the font configuration of every generated page.

A URL is a request address

`http://127.0.0.1:8000/product` contains a protocol (`http`), host (`127.0.0.1`, this computer), port (`8000`), and path (`/product`). The HTTP method is also part of the request. GET and POST can use the same path and trigger different code.

The application uses 8000 for `artisan serve`, 3307 for the XAMPP database, and 8080 as the recommended alternate Apache port for phpMyAdmin. A port identifies the service being contacted. The database port is not a page URL.

2. The full request journey

Opening a PHP application does not execute every file from top to bottom. Laravel loads its startup files, matches one route, runs middleware, and calls the method selected for that request. Other controller methods wait for other requests. JavaScript timers/events run later in the browser.

```
Browser sends method + URL + form data + cookies
    -> public/index.php
    -> vendor/autoload.php loads needed PHP classes
    -> bootstrap/app.php builds Laravel and loads route definitions
    -> web middleware: cookies, session, CSRF, shared validation errors
    -> route middleware: guest or auth, as configured
    -> matching controller / request class
    -> model, password broker, or other service when needed
    -> view HTML or redirect response
    -> browser displays HTML, loads CSS, runs JavaScript
```

The exact middleware stack is managed by Laravel. The source notes describe the steps relevant to our requests rather than claiming every internal framework method appears in this simplified diagram.

`public/index.php` calls `Request::capture()` to construct a request object and `$app->handleRequest(...)` to process it. `bootstrap/app.php` identifies `web.php` as the browser-route file. Composer's autoloader finds classes such as Product when code references them; loading a class does not itself insert a row.

Example: a guest opens Products

The route exists, but its `auth` middleware runs before `ProductController::index`. The visitor has no authenticated session, so Laravel returns a redirect to named `login`. The browser follows that redirect with a new GET request. It gets the Login view instead of receiving the product records.

Example: a signed-in user opens Products

The session identifies a user, middleware allows the request, and `index()` calls `Product::all()`. Eloquent reads rows. The controller passes `$products` to `products.index`, which extends `layouts.product`. Blade renders HTML; the browser then applies `public/css/style.css` and loads the custom public scripts.

A redirect creates a second request

A redirect says where the browser should go next. It does not render that next page in the same controller return statement. After saving a product, the POST response redirects to the GET list. This helps prevent ordinary refresh from resubmitting the original POST.

3. PHP syntax that appears in the project

Variables, values, and statements

```
$name = 'Apple';      // string: text
$quantity = 12;       // integer: whole number
$enabled = true;      // boolean: true or false
$description = null;  // no value
$price = '25.00';     // string representation of a decimal value
```

`$` marks a PHP variable. `=` assigns a value. A semicolon ends a statement. Single/double quotes enclose strings; double-quoted strings can interpolate variables. `null` is different from an empty string, zero, and false.

```
$data = ['name' => 'Apple', 'qty' => 12];
echo $data['name']; // Apple
```

This is an associative array: names are keys, and values are stored beside them. `=>` separates a key from its value. Square brackets construct/access arrays. A collection returned by Eloquent is an object with collection methods, not just the plain array above.

Functions and methods

A function is a named unit of work. A method is a function defined on a class. `view()` and `route()` are helper functions. `index()` is a controller method; `update()` is also an inherited Eloquent method used on a Product object. The same method name in two classes can mean different things.

```
function doubleQuantity($quantity) {
    return $quantity * 2;
}
$answer = doubleQuantity(3); // 6
```

`$quantity` in the definition is a parameter. `3` in the call is an argument. `return` sends the result back and ends that function call. It does not necessarily print anything. A controller returns a response object Laravel can send.

Classes, objects, namespaces, and imports

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Product extends Model {
    protected $fillable = ['name', 'qty', 'price', 'description'];
}
```

A class describes data and behavior. An object is an instance of a class, such as one loaded Product record. A namespace organizes class names; it is not a database name. `use` imports a class name for convenient reference.

`extends Model` inherits Eloquent behavior. `public` members are accessible to calling code. `protected` members are accessible within the class/its subclasses. `private` would restrict access to the declaring class. A protected `$fillable` is read by Eloquent internally; the browser does not submit a new fillable list.

```
Product::all();                 // class-level Eloquent call
$product->name;                 // object attribute
$product->update(['qty' => 15]); // method on this loaded object
ProductController::class;       // the fully qualified class name
```

`::` refers to a class-level member; `->` accesses an object. Eloquent implements some convenient calls through its internal forwarding/magic behavior. The `new` keyword constructs objects, for example `new Registered($user)`.

Type declarations and named arguments

```
public function store(LoginRequest $request): RedirectResponse
```

`LoginRequest` declares the expected parameter type. Laravel resolves that request object for the controller. `: RedirectResponse` declares the returned type. `: void` means no return value is supplied. `RedirectResponse|View` permits either type. `array<string, ...>` in PHPDoc is documentation for tooling, not runtime PHP syntax. A type declaration can help catch mistakes but does not replace validation.

```
route('product.index', absolute: false)
```

`absolute: false` is a named argument: it names the parameter instead of relying only on its position. The route helper returns a relative path here rather than a full URL. `__DIR__` is the current file directory; `dirname(__DIR__)` moves up one level. They avoid relying on the terminal's current folder.

Conditions, comparison, and boolean logic

```
if (auth()->check()) {
    return redirect()->route('product.index');
}
return redirect()->route('login');
```

Only the selected branch returns. `==` compares values with PHP's conversions; `===` compares value and type. `!` means NOT, `&&` means AND, and `||` means OR. `$this` refers to the current object, as in `$this->ensureIsNotRateLimited()`.

```
$word = $count === 1 ? 'product' : 'products';
```

The ternary operator selects the expression after `?` if the condition is true, otherwise the expression after `:`. `??` supplies a fallback for a missing/null value. In PHP, `.` concatenates strings; `+` is arithmetic addition.

Closures, callbacks, and chaining

```
Route::middleware('auth')->group(function () {
    // Routes in this callback receive auth middleware.
});
```

`function () { ... }` is an anonymous function, also called a closure. The route service calls it to register the grouped routes. Another closure runs after a password token is verified:

```
function (User $user) use ($request) {
    // use() makes the outer variable available inside this closure.
}
```

This closure's `use` is variable capture, unlike the class import at the top of a PHP file. PHP arrow functions such as `fn () => ...` are shorter callbacks with automatic by-value capture. JavaScript arrows have their own rules.

```
return redirect()->route('login')->with('status', 'Message');
```

Each `->` calls a method on the object returned by the previous expression. This is method chaining. Read it left to right: create a redirect, set its route, then flash feedback to the session.

Exceptions, interfaces, traits, and documentation comments

An exception interrupts normal execution. `throw ValidationException...` stops login and lets Laravel produce the validation redirect. JavaScript `try/catch` in the theme script catches blocked browser storage; it is a separate runtime.

An interface declares a contract a class implements. `MustVerifyEmail` is an interface Laravel checks before enforcing email verification. Our User does not implement it. A trait is reusable class behavior included with `use`, such as `HasFactory` and `Notifiable` inside User.

`//` explains PHP/JavaScript code. `/* ... */` is a block comment, needed in CSS. PHPDoc `/** ... */` may contain `@return`, `@var`, or `@throws` information for tools. JSON does not support either comment form. Comments explain the program; the program still runs when those comments are removed.

4. MVC and the responsibilities of each layer

The Product model is a PHP representation of database records. The Product controller coordinates requests. Views display values and collect input. Routes connect addresses/methods to the controller. Migrations define table structure.

Keep these questions beside the code:

Question	File to read
Which URL runs this method?	routes/web.php or routes/auth.php
What happens to submitted input?	Controller or FormRequest
Which table holds this record?	Model convention and migration
Where is the HTML produced?	Matching Blade view and layout
Why does this element look this way?	CSS class selector
What happens on click in the browser?	JavaScript listener or form submission

`Product::all()` does not enforce ownership. It retrieves all products. Auth middleware ensures a visitor has logged in, but currently every logged-in user can operate on shared product records. Adding owner/admin rules would require additional relationships/policies and is outside the implemented lab scope.

5. Product CRUD, with one concrete record

Read the product list

GET `/product` passes auth and calls `ProductController::index()`. Eloquent reads the products table and returns a collection. `compact('products')` creates `['products' => $products]`; the view receives the variable under that key. `@forelse` renders each Product, or an empty row when none exist.

Create Apple

GET `/product/create` calls `create()`, returning the form. The form's `action` is the URL generated by `route('product.store')`. POST includes name, qty, price, description, and CSRF token. `store()` validates before calling `Product::create`. If validation fails, execution never reaches the insert.

```
$data = $request->validate([
    'qty' => 'required|integer|min:0',
]);
```

Input quantity `12` passes. `-1` fails min, `1.5` fails integer, and an absent quantity fails required. HTML attributes also check these values, but the server must enforce its rules because a caller can modify browser HTML.

The real controller validates all four fields. `$fillable` permits their mass assignment. Eloquent inserts a row, the database assigns its ID, and timestamps are managed. The response redirects to the list with a flash success message. JavaScript removes that banner after five seconds; it does not undo the insert.

Update Apple quantity from 12 to 15

The Edit URL includes the record ID. `{product}` matches `Product $product`, so route model binding loads the record. A missing ID normally returns 404. The edit view displays `old('qty', $product->qty)`: previous submitted input wins after an error; otherwise use the stored quantity.

The form submits POST with hidden `_method=PUT`. Laravel matches the update route, validates, and calls `$product->update($data)`. The record ID is preserved; `updated_at` changes. Opening `edit()` alone does not save anything.

Delete a disposable product

The form targets `product.destroy`, contains CSRF, and uses `_method=DELETE`. Browser `confirm()` returns true for OK and false for Cancel. That return value determines whether this submission proceeds. The controller permanently removes the selected row through `$product->delete()` and redirects.

Confirmation helps prevent accidental clicks. It is not authorization. The server still requires auth, but it has no per-product ownership policy.

6. Database structure and persistent values

Table	Purpose	Main connection
products	Product records	Product model and ProductController
users	Accounts and password hashes	User, registration, login
password_reset_tokens	Reset token hashes and creation time	Password broker
sessions	Session payload and activity	Session web guard
cache / cache_locks	Database-backed cache and locks	Configured cache services
jobs / job_batches / failed_jobs	Optional queued work	Queue services
migrations	Which schema migrations ran	Artisan migrate

`products` contains id, name, qty, price, optional description, created_at, and updated_at. `decimal(8,2)` allows eight total digits, two after the decimal: up to 999999.99 in the positive range. It stores a fixed decimal rather than an approximate floating-point value. Display formatting and validation are separate.

`users.email` has a database unique constraint as well as registration validation. `password` contains a hash. `remember_token` supports remembered authentication. `email_verified_at` can hold a timestamp, but a column alone does not enforce verification. The sessions `user_id` is nullable for guest sessions and indexed; this migration does not add a foreign-key constraint to that field.

`up()` creates schema and `down()` reverses it. Laravel records successful migrations, so another `migrate` says Nothing to migrate. Editing a migration already run does not automatically alter the database. A schema change should normally be a new migration. Avoid `migrate:fresh` on your real database because it drops tables and their data.

SQL conceptual examples:

```
SELECT * FROM products;
UPDATE products SET qty = 15 WHERE id = 6;
```

These explain what reading/updating means. Your controller uses Eloquent rather than manually interpolating form input into SQL. Eloquent/query-builder value bindings reduce SQL injection risk for these calls. Raw SQL expressions and unvalidated column names would need their own care.

Database files/exports and the real `.env` are excluded from Git. Migrations in Git can recreate table structure, not your current MySQL records. Back up the database separately before changes.

7. Registration, step by step

GET `/register` is a guest route. `RegisteredUserController::create()` returns `auth.register` inside `x-guest-layout`. POST `/register` calls `store()`.

The rules require name, lowercase valid email, unique email, and confirmed password. `Rules\Password::defaults()` uses the configured default rule; without customization its minimum is eight characters. Name/email limits are 255. `confirmed` compares password against `password_confirmation`.

```
'password' => ['required', 'confirmed', Rules\Password::defaults()]
```

The rule list is an array rather than a pipe-separated string because it also contains a Password rule object. `unique:'.User::class` concatenates the unique rule prefix with the model class name. The model identifies the users table.

After validation, `User::create` saves name, email, and `Hash::make(password)`. A password hash is a one-way representation checked by a hashing service. It is not encryption to be decrypted later. The User `hashed` cast recognizes a valid already-hashed value, so this code does not intentionally double-hash it.

`event(new Registered($user))` lets listeners react to registration. `Auth::login` signs in that new account. This controller still redirects to Dashboard. Your normal login controller defaults to Products. The guide documents the actual two flows rather than assuming they share a redirect.

8. Login, remembered authentication, and logout

Login validation and credential checking

The form posts email/password/remember to `/login`. Laravel resolves LoginRequest and checks its rules before calling the controller's store method. Required email/password rules ensure suitable input; they do not prove the password is correct. `$request->authenticate()` performs that credential check.

```
Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))
```

`only` selects credential keys. `boolean` interprets the checkbox. The web guard uses the configured Eloquent User provider to find the account and verify its password hash. A failed attempt increments the limiter and throws an email validation error. Five failures for the normalized email/IP combination can cause a wait; success clears that counter. This is distinct from reset throttling.

After success, the controller regenerates the session ID, reducing session fixation risk. `redirect()->intended(...)` returns to a protected page attempted before login, otherwise `/product`. Merely rendering Login does not authenticate.

Remember me

Checking it asks Laravel to issue remembered authentication using its cookie/token mechanism. It does not store the readable password, automatically check every future form, or add administrator rights. `old('remember')` only keeps the checkbox choice when a form fails. Removing/clearing cookies affects remembered login; logging out ends the current authentication flow.

Logout

The product layout has a POST logout form with CSRF. The controller calls:

```
Auth::guard('web')->logout();
$request->session()->invalidate();
$request->session()->regenerateToken();
return redirect()->route('login');
```

Read the sequence: log out the guard, discard session data, rotate the CSRF token, redirect to Login. A GET link alone should not perform logout. If you then request Products, auth middleware asks for login again. It does not delete your user account or product records.

9. Password recovery and SMTP

Forgot Password requests an email. Reset Password changes the password using a valid token. Profile password updates require the current password. Password confirmation records recent proof without changing it. These are separate flows.

Request the email

POST `/forgot-password` validates an email and calls `Password::sendResetLink($request->only('email'))`. The password broker finds the User, checks its throttle, creates a reset token, stores its hash, and notifies the user. The email link contains the token needed by the reset form.

`config/auth.php` has expire 60 (minutes) and throttle 60 (seconds). A quick second request can show Please wait before retrying. This protects against repeated mail requests; it is not a broken form. Generating a new token replaces the previous stored request for that email.

The controller compares a status to `Password::RESET_LINK_SENT`. Success flashes a translated status; failure returns old email with an error. `__($status)` is Laravel's translation helper, not the JavaScript underscore convention.

Send through the configured mailer

`config/mail.php` reads `MAIL_MAILER`, `MAIL_SCHEME`, host, port, username, password, and sender settings from `.env`. This app's SMTP config uses `MAIL_SCHEME`; adding MAIL_ENCRYPTION alone would not change this configured key.

Setting	Mailtrap sandbox example	Gmail example
MAIL_MAILER	smtp	smtp
MAIL_SCHEME	smtp	smtp
MAIL_HOST	sandbox.smtp.mailtrap.io	smtp.gmail.com
MAIL_PORT	2525	587
MAIL_USERNAME	Sandbox credential	Full sender Gmail address
MAIL_PASSWORD	Sandbox credential	Google App Password

These are configuration examples without secrets. SMTP on these ports can upgrade through STARTTLS; keep TLS certificate verification enabled. Gmail's App Password is separate from the Laravel user's login password and from the normal Google account password. Mailtrap captures messages in its sandbox; Gmail sends to the registered recipient's actual inbox. One chosen SMTP configuration is active at a time in the current application.

Submit a new password

GET `/reset-password/{token}` passes Request to the view. The hidden token field uses `$request->route('token')`; the email starts from the query string. The guest middleware may redirect a signed-in browser away, which explains why you needed to log out to open this form.

POST `/reset-password` validates token/email/confirmed password and calls `Password::reset`. Only a valid, unexpired token reaches its callback. The callback hashes the new password, rotates `remember_token`, saves the

User, and fires PasswordReset. Successful reset returns to Login with a status; it does not by itself take the user through a normal login in this controller. The new password then works through the standard login flow.

The callback uses `forceFill` with explicitly selected keys. This bypasses mass-assignment restrictions deliberately; passing every browser key to forceFill would be unsafe. A token is not a permanent login credential and should not be included in public screenshots.

10. Authentication, authorization, and other protections

Authentication checks who the visitor is. Authorization checks whether that visitor may perform a particular action. Our product routes require login, but there are no admin roles or owner-only policies. User A can currently manage the same products as User B. A signed-in session alone does not make either admin.

Mechanism	Checks/protects	Does not provide
auth middleware	Signed-in session	Owner/admin permission rules
guest middleware	Signed-out access to auth forms	Product authorization
validation	Submitted values	Identity or SQL permissions
fillable	Allowed mass-assignment keys	Value validation
CSRF token	Forged session-based form requests	Login or user roles
password hash	Password storage/checking	Hiding visible browser inputs
Show/Hide button	Browser display of typed password	Encryption or server security
escaped Blade output	HTML interpretation of displayed text	Database validation
bound database values	These queries resist SQL injection	Safety of arbitrary raw SQL
.env exclusion	Keep local secrets out of Git	Protection if files are leaked elsewhere

Email-verification handlers exist, but User does not implement MustVerifyEmail. The product group applies auth only. The generated Dashboard verified middleware does not establish that verification is enforced for this current model.

Profile account deletion is a generated feature requiring the current password. Products have no owner relationship, so the generic deletion template's wording about deleting all resources should not be interpreted as a product cascade. The guide maps this boundary rather than claiming unimplemented behavior.

11. Blade layouts, directives, and components

Two layout systems in this project

Product views use inheritance:

```
@extends('layouts.product')
@section('content')
    <h1>Products</h1>
@endsection
```

The layout uses `@yield('content')`. Dot notation maps a name to folders: `products.index` is `resources/views/products/index.blade.php`.

Auth views use components:

```
<x-guest-layout>
    <h1>Welcome back</h1>
</x-guest-layout>
```

GuestLayout::render selects `layouts.guest`; the inner markup becomes `$slot`. Dashboard/Profile use x-app-layout with `$slot` and sometimes a named `$header`. `@extends('layouts.app')` does not supply the component slot, which caused the earlier Undefined variable $slot error. Both approaches work when the calling page and layout agree on how content is supplied.

Output, loops, errors, and form protection

`{{ $product->name }}` displays escaped data. Entered `<script>` text is treated as text rather than inserted as executable HTML through this syntax. `{!! ... !!}` is unescaped output and is not used for our user-entered product values.

`@if`, `@else`, and `@endif` choose blocks. `@foreach` iterates; `@forelse` adds `@empty` behavior. `@error('email')` exposes `$message` for that field. `$errors` can contain multiple bags; profile password/account forms use named bags to keep unrelated errors apart.

```
<form method="POST" action="{{ route('product.update', $product) }}">
    @csrf
    @method('PUT')
</form>
```

HTML forms submit GET or POST. `_method` lets Laravel route a POST as PUT/DELETE. `@csrf` inserts a hidden `_token` tied to the session. Middleware verifies it. `old()` restores safe previous input after errors. Passwords should never be restored into HTML. `@auth` renders content for authenticated users.

`@include` inserts a partial, such as profile form markup. Anonymous components under views/components use `@props` to declare input values and `$attributes` to forward/merge attributes. Named slots such as trigger/content supply dropdown markup. `@js` produces safely encoded JavaScript data. Blade `@php` blocks contain our hidden `//` comments; bare `//` in ordinary HTML would appear as page text.

HTML form attributes

Attribute	Purpose
name	Submitted field key the server receives
id	Label/DOM target; not by itself a submitted key
for	Connect a label to the matching input ID
type	Email, password, checkbox, number, or hidden behavior
value	Initial displayed/submitted value
required/min/max/step	Browser constraints, rechecked server-side
autocomplete	Tell password managers/browser the field purpose
autofocus	Initial focus, not a validation rule
class	CSS/DOM selector hook, not a database field

Attribute	Purpose
aria-hidden	Hide decorative icons from assistive tools
aria-label/pressed/controls	Name/state/relationship for interactive controls
defer	Run the linked script after HTML parsing

Use `type="button"` for Show/Hide so it does not submit the form. A normal submit button intentionally sends it. Hidden fields are invisible in the layout but still inspectable in the browser; never treat them as trustworthy secrets.

12. CSS, glass surfaces, and the font

A selector identifies elements, braces group declarations, and each declaration has a property/value separated by `:` and ended by `;`.

```
.auth-card {
    max-width: 460px;
    padding: 36px;
}
```

The class selector begins with `.`. A descendant selector such as `.light-theme .auth-card` selects a card inside a light-theme ancestor. `:hover` and `:focus-visible` are states; `::after` is a pseudo-element. Our navbar divider is drawn separately so the theme button's width does not shift when toggled.

The cascade considers origin/layer, specificity, and order. A more specific selector can override a later weak selector. This is why the password wrapper padding rule repeats the input exclusions: it must beat the earlier generic input padding. A long file of overrides is harder to read; use its section comments and CODE_MAP to find the relevant rule before adding another duplicate.

`box-sizing:border-box` includes padding/borders inside declared widths. Padding is internal space; margin is outside. `max-width` limits desktop width while `width:100%` fills the available wrapper. Flexbox aligns children and can wrap them when space is limited. `gap` separates children without manual margins.

The table has `min-width` for readable columns, while its wrapper has `overflow-x:auto`. Scroll inside that wrapper on small screens. The media query at 600px reduces padding/type. `scrollbar-gutter:stable` reserves scrollbar space to reduce theme-switch layout shifts. The form is not supposed to be squeezed into six tiny table columns.

Glass combines transparent `rgba` values, background gradients, backdrop blur, bright inner edges, rounded corners, and offset shadows. Alpha 0 is transparent; 1 is opaque. Stacking several pale backgrounds can make the result look solid. `inset` draws a shadow inside the element. `saturate()` changes the backdrop color intensity, not text saturation. This approximates glass on the web without Apple's native refraction renderer.

The primary action is pearl white in dark mode and charcoal in light mode. Delete remains coral. Error/success colors also carry readable messages so color is not the only signal. Reduced-motion rules remove the button transition.

Inter loads through Google Fonts in custom layouts; fallbacks apply if the download fails. Input/button `font:inherit` copies the parent font. Generated Dashboard/Profile still use their Vite/Tailwind font setup. A font-family rule selects a font; the link in the layout actually makes the hosted font available.

13. JavaScript and SVG, explained through the three scripts

JavaScript syntax and browser objects

`const` prevents reassignment of a variable binding; it does not freeze every property on an object. `let` permits reassignment. `document` is the DOM API for this page. `querySelector` returns the first matching element or null; `querySelectorAll` returns a list-like NodeList. `forEach` runs a callback for each match. JavaScript methods use `.`, not PHP's `->`.

`===` is strict equality. The ternary operator chooses one expression. String concatenation uses `+` in our JavaScript. `try/catch` handles exceptions. DOM updates do not automatically send a request to the server or update MySQL.

theme.js

The script finds `theme-toggle`, reads a saved value, and calls setTheme. The function toggles light-theme on body and updates accessible button state. Its click listener chooses the other mode and saves localStorage. A blocked storage exception leaves the current-page toggle usable. LocalStorage is browser storage; it is separate from Laravel sessions and should not hold our passwords.

password-visibility.js

The selector finds password inputs inside auth-input wrappers. For each input, the script creates a non-submit button, reserves right padding, and appends it. Clicking switches `input.type` between password and text, changes Show/Hide text, and updates accessible state. The field name/value/autocomplete remain intact. Each confirmation field has its own toggle. The typed password is still sent through the same form and is hashed on the server.

product-messages.js

The script selects success-message. If no banner exists, it does nothing. Otherwise `setTimeout` schedules a callback for 5000 milliseconds (five seconds). `remove()` takes the banner out of the DOM. Validation errors use different classes and stay visible. Removing a DOM element does not delete a database record.

SVG outline icons

`svg` creates a vector drawing. `viewBox="0 0 24 24"` sets its coordinate space. `path d` contains move/line/curve commands; rect/circle describe shapes. `fill="none"` creates outlines, `stroke="currentColor"` inherits the text color, and stroke-width controls line thickness. Icons are Lucide-like drawings, not React components. `aria-hidden` avoids announcing decorative shapes twice.

Alpine in generated pages

`x-data` creates local UI state; `x-show` controls visibility; `x-on:click` or `@click` listens to events. `$dispatch` sends a custom event, `$watch` observes state, and `$el` refers to the component element. Navigation dropdowns and the profile deletion modal use Alpine. The custom theme/password controls use plain JavaScript instead. These tools do not authorize database changes.

14. Configuration, builds, and dependencies

`.env` contains local configuration, including credentials. `config/*.php` maps environment values into Laravel settings. `config('mail.default')` reads the resolved setting; `env()` belongs mainly in config definitions. Cached configuration can retain old values until `artisan config:clear` is run. Do not print the entire resolved configuration when sharing errors.

Composer reads composer.json and composer.lock, installs PHP dependencies into vendor, and creates autoload metadata. A lockfile records exact resolved versions. `require-dev` includes development/test packages such as Breeze/PHPUnit. Breeze generates source code; installing its package alone does not create all views/routes.

npm reads package.json/package-lock.json and installs Node packages. Tailwind scans templates; PostCSS/Autoprefixer help produce CSS; Vite builds entry assets. The Laravel vite directive reads the manifest or development-server information. `npm run build` creates a reusable build; `npm run dev` keeps a live build server. Custom files in public/css and public/js are served directly and do not need to be compiled by Vite after every change.

The default composer setup script includes migrations; do not run broad setup commands blindly against an existing database. Use the documented commands and inspect what the scripts do. Framework dependencies should be reinstalled through tools rather than edited to hide SMTP errors.

15. Tests, evidence, and real limitations

PHPUnit runs assertions through artisan test. A feature test boots Laravel and simulates HTTP requests; a unit test can examine a smaller piece. RefreshDatabase resets the test database between relevant tests. phpunit.xml selects in-memory SQLite, array sessions/cache/mail, so these tests do not delete your MySQL products or send actual Gmail mail. Always verify this isolation before running unfamiliar database-reset tests.

UserFactory supplies fake test users. `$this->get`, `post`, and `actingAs` simulate requests/authentication. `assertRedirect`, `assertSessionHasErrors`, `assertGuest`, and `assertAuthenticated` verify outcomes. Notification::fake captures reset notifications so tests can inspect token flow without emailing real users.

Generated test expectations must follow customized routes. Our normal login now defaults to Products, logout to Login, and home redirects guests. Registration still defaults to Dashboard. Updating expected destinations does not weaken credential checks; separate assertions still check guest/authenticated state.

Tests of email-verification handlers do not enable enforcement. Tests of generated controllers do not certify the final mobile layout. A successful SMTP test does not mean every provider account has the same settings. Screenshots show one state; a recorded user action or automated assertion establishes behavior.

16. Run, inspect, and explain the application

```
cd "C:\Users\Yestin Paraguya\Downloads\Paraguya_CRUD_Lab2\app-crud"
& "C:\xampp\php\php.exe" artisan serve
```

Start XAMPP MySQL and keep the server terminal open. Open http://127.0.0.1:8000/. Guests reach Login; signed-in users reach Products. VS Code may be closed if the server runs in a separate PowerShell window.

```
& "C:\xampp\php\php.exe" artisan route:list -v
& "C:\xampp\php\php.exe" artisan migrate:status
& "C:\xampp\php\php.exe" artisan view:clear
& "C:\xampp\php\php.exe" artisan config:clear
& "C:\xampp\php\php.exe" artisan test
```

Read each command's purpose: list routes/middleware, inspect completed migrations, clear compiled view cache, clear resolved config cache, run tests. None of the first four intentionally deletes product rows. Ctrl+F5 refreshes browser assets.

Open phpMyAdmin through the configured Apache port, select the lab database, then Browse/Structure on products or users. Passwords should appear as hashes, not their original text. Never include hashes, reset URLs, cookies, or SMTP credentials in a public demonstration screenshot unnecessarily.

To explain code, start with an action: "This form submits POST /product, the auth group requires a session, store validates input, Product inserts the row, and the redirect returns the list with feedback." Then point to the files that prove each clause. That is more useful than memorizing all methods in file order.

17. References for further study

- PHP classes and objects
- Laravel request lifecycle
- Laravel routing
- Laravel Blade
- Laravel validation
- Laravel authentication
- Laravel password reset
- Laravel email verification
- Laravel Eloquent
- MDN JavaScript guide

Examples throughout this guide are tied to the checked application source. The course's Laboratory 3 PDF supplies the activity questions and deliverables.

Laboratory 3 guide-question answers

The questions below follow Section V of the Laboratory 3 PDF. Use the short answers when speaking, then point to the example/code when asked to explain more. They describe the actual application; they do not claim admin roles or required email verification have been implemented.

Use Instructor code locator for exact source files, current line numbers, code excerpts, and likely follow-up questions. That appendix is the final part of the printable handbook.

1. What is Laravel Breeze, and what problem does it solve?

Breeze is a Laravel authentication starter package that generates routes, controllers, request validation, views, and tests for common account features. It provides registration, login, logout, password recovery, and profile-related code, so the developer can inspect/customize these flows rather than write every piece from an empty project.

In this application, Composer installed Breeze, then `breeze:install` generated the Blade/Alpine scaffold. npm installed frontend tools and the build compiled their assets. Breeze did not automatically protect our existing Product routes; we added the auth middleware group separately.

Point to `routes/auth.php`, `app/Http/Controllers/Auth/`, and the auth views. Explain the difference between installing the package and generating its scaffold. Keep expected form names/routes when changing its appearance.

Short answer: "Breeze generates the account forms and authentication code we need, including login, registration, and password recovery. We customized the views and separately protected the existing product routes."

2. What is the difference between registration and authentication?

Registration creates an account. Authentication verifies that someone using an existing account supplied valid credentials and establishes their signed-in state. Our registration handler also signs in the newly created user, but those remain two different operations in its code.

Example: entering a new email/password on Register creates a users row with a password hash. Later, submitting Login checks that saved account and hash; it does not create another users row. An email already registered is rejected by the unique-email rule on Register, while the same email is expected on Login.

Point to `RegisteredUserController::store()` and `LoginRequest::authenticate()`.

Short answer: "Registration creates the account; authentication checks the account credentials when the user signs in. Registration also logs the new user in in our Breeze flow."

3. What is the purpose of the auth middleware?

auth middleware checks the session's authentication before allowing the request to reach its protected controller. For our HTML product routes, a guest is redirected to named login. A valid signed-in session lets the request continue.

The group in web.php contains all six product routes, including POST store, PUT update, and DELETE destroy. Protecting only the list page would leave those other actions accessible through direct requests.

Point to `Route::middleware('auth')->group(...)` and show route:list with `-v`. Then demonstrate requesting `/product` while logged out.

Short answer: "auth middleware requires a signed-in session before the product controller runs. Guests are redirected to Login, and all six CRUD routes are inside the protected group."

4. Why should Product CRUD routes be protected from unauthenticated users?

Product records are application data. If visitors could call store/update/destroy without authenticating, they could create or change records without an account. Protecting every action establishes the lab's login boundary, including direct requests that do not use the visible buttons.

This boundary does not decide ownership or administrator privileges. The current Product model has no user_id, and there are no policies checking who owns a product. All authenticated users can still manage the shared list.

Point to the auth group and the lack of an owner relationship/policy. Explain that hiding a Delete button alone is not server-side protection.

Short answer: "Login protection prevents guests from reading or changing product records through our CRUD endpoints. We would need authorization rules as well to restrict particular logged-in users or owners."

5. What is the role of SMTP configuration in password recovery?

SMTP configuration tells the mailer which email server to contact, on which port, and which credentials/sender identity to use. Laravel's password broker creates the reset request and notification; the SMTP transport delivers that email. SMTP itself does not validate the reset token or save the new password.

We tested Mailtrap's sandbox and Gmail one at a time. Mailtrap captured the email in a test inbox; Gmail delivered it to the registered account's inbox. The email link opened the form with a token, which NewPasswordController passed to the broker for verification before updating the account password.

Point to `config/mail.php`, `PasswordResetLinkController`, and `NewPasswordController`. Show the delivered email while hiding its reset URL.

Short answer: "SMTP makes the reset email deliverable. Laravel creates and verifies the reset token, while the configured mail server sends the link to the user."

6. Why should email credentials be kept in the .env file?

They vary by developer/deployment and should not be hard-coded into a controller or committed to a public repository. The private `.env` supplies credentials to `config/mail.php`; a safe `.env.example` can show field names without secrets. The repository ignore rules exclude the real environment file.

An `.env` is not magic encryption. A leaked file or printed configuration still exposes its values. Do not include it in screenshots, paste it into chats, or serve the project root as a public web directory. Application keys and database passwords need the same care. If a credential is published, remove/revoke/rotate it through its provider; deleting it from the current Git file does not erase history.

Point to `config/mail.php` and `.gitignore`, without displaying actual credentials.

Short answer: "The private environment file separates sensitive deployment values from source code. We commit only an example with no secrets, and config/mail.php reads the real values locally."

7. What is the difference between authentication and authorization?

Authentication verifies identity or signed-in state. Authorization decides whether that identified user is allowed to do a particular action. A user may be authenticated yet lack permission to delete someone else's record.

Example: both a clerk and manager may log in successfully. A separate policy could allow only the manager to delete products. That clerk/manager rule is an example of a possible extension; our code currently has neither role nor policy.

Point to `auth` in web.php for implemented authentication. Explain that policies, gates, or explicit ownership checks would enforce additional authorization. `$fillable`, CSS themes, and the Remember me checkbox are not permission checks.

Short answer: "Authentication checks who is signed in. Authorization checks what that user may do. Our auth group supplies the first check; role/ownership rules would be additional work."

8. Does placing a route inside the auth middleware group automatically make a user an administrator? Explain your answer.

No. The group checks that the visitor is authenticated using the configured guard. It does not add an administrator flag, assign a role, or compare the user against a permission policy. Any signed-in user can reach our Product routes.

To make certain actions administrator-only, the application would need a real role/permission representation and a server-side authorization check, tested with users who are allowed and users who are denied. Merely naming a route admin or making its button invisible does not implement this check.

Point to User's fields and the auth-only product group. State the current limit without claiming a role feature was added by Breeze.

Short answer: "No. auth only requires login. Administrator privileges need separate data and authorization rules, which our current application does not have."

Required deliverables from the PDF

1. Screenshot of the Breeze login page (our customized Breeze view is acceptable evidence of its UI).
2. Screenshot of the registration page.
3. Screenshot showing a successfully registered user.
4. Screenshot of Product CRUD after login.
5. Screenshot of the login redirect when a logged-out user requests `/product`.
6. Screenshot of a password-reset email or a successful email test.
7. Copy of modified `routes/web.php`.
8. Short explanation of how auth middleware protects Product CRUD.

For item 8, an example explanation is: "All six product routes are in the auth middleware group. Laravel checks the web session before calling ProductController. A guest requesting a product route is redirected to Login; a signed-in user may continue. This protects the endpoints from unauthenticated access, while additional role/ownership authorization is outside our implemented scope."

The PDF states that face-to-face checking includes questions. Practice pointing to code and tracing a request rather than only reciting the short answers.

Practice, checks, and demonstration

Use disposable products and test accounts. Keep real passwords, SMTP values, session cookies, and reset tokens out of notes/screenshots. Exercises below ask you to predict behavior before checking the answer.

A reading schedule you can repeat

Before session 1, read TERMS_AND_CONCEPTS.md sections 1-3: browser/server, requests/methods, and routes/middleware. Pick one definition and point to its use in web.php. Read its remaining categories as you encounter those terms in later sessions.

Session	Read and open	Explain without looking
1	Guide chapters 1-3; public/index.php and bootstrap/app.php	Browser/server difference and one request journey
2	Chapters 4-6; web.php, ProductController, Product, migration	Add Apple, update its quantity, then delete a test row
3	Chapters 7-8; auth.php, LoginRequest, login/registration controllers	Registration vs login and where a session changes
4	Chapters 9-10; reset controllers, mail/auth config	Token recovery and authentication vs authorization
5	Chapters 11-13; layouts/views/public scripts	Slot vs section, field keys, theme/show-hide/timer
6	Chapters 14-16; build/tests; Lab 3 answers	Run commands, demonstrate evidence, answer follow-ups

Spend enough time on a session to write a short explanation and trace one concrete value. Repeat the prediction exercises the next day, then a few days later and a week later. Progress comes from recalling and applying the explanation, not from assuming one long read will make every API permanent knowledge.

Predict the behavior

1. Assignment and types

```
$qty = 3;
$nextQty = $qty + 2;
```

What are the values afterward? Does qty change?

Answer: qty remains 3; nextQty is 5. The expression computes a new value assigned to another variable. Assignment to nextQty does not automatically change qty.

2. An associative array

```
$data = ['name' => 'Apple', 'qty' => 12];
```

What does data['qty'] return? What is the purpose of =>?

Answer: 12. The arrow associates the key qty with that value. This array is not yet a saved Product; saving requires a model operation.

3. Browser required vs server required

Someone removes required from the Name input in browser Inspect. Does the server accept an empty name?

Answer: no. ProductController's required validation still runs. Browser constraints help the user, while server validation enforces input for direct/modified requests.

4. Fillable vs validation

Would adding qty to fillable reject -5 by itself?

Answer: no. Fillable permits that key for mass assignment. min:0 validates its value. The database integer type alone is also not the complete input rule.

5. View name vs file path

Which file is selected by view('products.edit')?

Answer: resources/views/products/edit.blade.php. The controller must also pass the Product object used by that view.

6. Product object vs collection

Why does index loop over products while edit reads product->name directly?

Answer: index receives a collection of records. edit receives one bound record. The singular/plural variable names communicate that distinction; Laravel does not infer it just from the English names.

7. When does create happen?

Does opening /product/create insert a row?

Answer: no. GET create returns a form. POST store validates and inserts.

8. Why a hidden PUT field?

The update form says method POST. How does it match a PUT route?

Answer: the hidden _method generated by @method('PUT') tells Laravel to treat the request as PUT. Browsers do not natively submit this HTML form as PUT.

9. Guest can see a Delete URL

If someone knows a product's ID, does knowing its URL bypass auth middleware?

Answer: no. Middleware evaluates the request/session before the controller. Knowing an ID does not grant login. A signed-in user still has access to the shared records because we have no ownership policy.

10. Intended redirect

A guest opens /product/create, logs in, and there is no intervening change to the saved intended destination. Which page should open?

Answer: Add Product, because intended redirects to the originally requested protected page. The fallback Products page is used when there is no destination.

11. Checkbox value

Does Remember me save the user's plain password in localStorage?

Answer: no. LoginRequest passes its boolean to Laravel's guard, which uses its remembered authentication mechanism. LocalStorage here stores only theme choice.

12. Password visibility

After Show changes input.type to text, does hashing stop?

Answer: no. It changes browser presentation. The form submits the same field, and registration/reset controllers hash the password on the server.

13. Reset cooldown

Why does Please wait before retrying appear after a second quick reset request?

Answer: config/auth.php's password broker throttle is 60 seconds. This is not the token expiration, which is 60 minutes. Keep the wait protection.

14. Password reset token

Is the hidden token input optional because the user cannot see it?

Answer: no. The broker needs it to verify the reset request. Invisible is not the same as unused or confidential from the user.

15. Reset while already signed in

Why might an email link open Dashboard/another authenticated destination instead of the reset form?

Answer: reset forms are guest routes; signed-in users may be redirected away. Log out or use a private browser window to inspect the reset form as a guest.

16. Session clearing

Does logout remove products from MySQL?

Answer: no. It clears authentication/session information. Product deletion is a separate controller/model operation.

17. Component mismatch

Why did using @extends('layouts.app') cause an undefined slot error?

Answer: that layout expects component content as slot. Inheritance provides sections for yield. Product pages now extend layouts.product; auth uses x-guest-layout.

18. Flash vs timer

What does the five-second timer remove, and who created the success message?

Answer: JavaScript removes the rendered success banner. ProductController flashed the message into the session during a redirect after the database operation.

19. Roles

Does auth middleware make a newly registered user an administrator?

Answer: no. It checks signed-in state. Our User data and routes do not establish an administrator role.

20. Verification scaffold

The verify-email routes exist. Does that prove email verification is required?

Answer: no. User currently does not implement MustVerifyEmail, and the product group requires only auth. Generated files can exist without full feature enforcement.

Safe hands-on practice

1. Create a disposable product, note its ID, edit its quantity, and verify the ID remains the same. Explain which methods handle display and saving.
2. Submit invalid product input and locate the exact rule and view error block.
3. Log out and request a protected URL. Record the redirect before signing in.
4. Use an incorrect login password once, then the correct one. Explain the two checks: field validation and credential authentication.
5. Try registering with an existing email and mismatched confirmation. Explain why neither attempt should create a user.
6. Request a reset email, open the newest link while logged out, and reset a test account password. Point to the broker, hidden token, hash, and login redirect.
7. Switch themes and Show/Hide password. Explain why MySQL data does not change.
8. Shrink the browser for all four auth pages and list/add/edit product pages. Check labels/buttons/errors remain inside their panels and table scrolling happens inside its container.

Do not disable TLS, remove auth/CSRF, or reset the real database as a shortcut. If practicing temporary validation changes, restore them immediately afterward.

Demonstration order

1. Open `/` while logged out; show Login.
2. Show Register and its validation. For a required success screenshot, register a disposable new user; explain that registration currently opens Dashboard.
3. Show guest `/product` access redirecting to Login.
4. Sign in; show Products and one full CRUD round with a disposable product.
5. Show incorrect login rejection, logout, and the guest-access check again.
6. Show the delivered reset email with its URL/token obscured, then explain/reset through the current styled form.
7. Point to web.php's six auth-protected routes and auth.php's guest/account routes.
8. Answer the eight guide questions using the application examples.

Keep the PDF's eight deliverables in LAB3_ANSWERS.md beside the captured evidence. Avoid filling screenshots with credentials or personal data that is not needed to demonstrate the task.

What was demonstrated, and what still needs manual checking?

The student demonstrated registration/login/logout, guest redirect, authenticated CRUD, and successful reset email/password use through Mailtrap and Gmail. Login rejection after the UI change was shown. Desktop screenshots show the customized Login/Register/Forgot views in both themes; icons were added afterward.

Registration validation after the latest customization, final Reset Password screenshots, long-term Remember me behavior, and narrow-screen checks for the new auth/logout/password-viewer UI need explicit manual evidence. Automated tests can cover server rules and redirects, but not replace those visual checks. The final verification report is in VERIFICATION.md.

Troubleshooting by layer

Symptom	First place to inspect
Could not open input file artisan	Terminal folder; run inside app-crud
Route not defined	routes/web.php/auth.php and route:list
Undefined variable slot	Calling layout style: component vs section
View not found	Dot-name path under resources/views
Database refused connection	XAMPP MySQL and DB host/port 3307
Nothing to migrate	migrate:status; already-run migrations are recorded
Password reset cooldown	Broker throttle; wait instead of repeatedly clicking
SMTP 535	Provider username/full App Password, no masked placeholder
SMTP/connection/TLS error	Host/port/network/certificate settings; never share secrets
New password not accepted	Correct account, matching confirmation, newest valid link
Styles not changed	Correct public stylesheet; Ctrl+F5; stylesheet order
Vite manifest missing	npm install/build from app-crud
Inter not visible	Hosted font/network; inspect rendered font/fallback
Private value in screenshot	Remove it before publishing; rotate leaked credentials if needed

Independent mastery checks

After a day, explain Add Product without the guide. After a few days, explain Login and a rejected guest request. After a week, explain reset email versus reset submission, then draw the route/controller/model/view connections from memory. Reopen the source and correct any missing steps.

For each method you read, answer: who calls it, what input it receives, which checks can stop it, what data it reads/writes, and what response it returns. For each visual script, answer: which DOM element it selects, which event/timer runs it, which state changes, and whether any request reaches the server.

Syntax and API reference used by this application

Use this reference after reading a request walkthrough. API means the names and contracts code calls, such as Laravel methods or browser DOM methods. The examples illustrate expressions; they are not extra features added to the project.

For the meaning of a term such as middleware, request, model, or POST, read Terms and concepts first. This reference explains the written code expressions used to work with those concepts.

PHP language expressions

Expression	Meaning and example
`<?php`	Start PHP code in a PHP source file
`$name`	Variable; `$name = 'Apple';` assigns a string
`;`	End a statement; block braces group several statements
`[]`	Construct/index arrays; `['qty' => 12]` or `$data['qty']`
`=>`	Map a key to a value inside an array; also appears in arrow functions
`->`	Read a property or call a method on an object: `$product->name`
`::`	Class-level member/reference: `Product::all()` or `Product::class`
`.`	Concatenate strings: `'unique:'.User::class`
`=`	Assignment; different from comparison
`==` / `===`	Value comparison with conversions / strict value-and-type comparison
`!`	Negate a boolean result, such as failed `Auth::attempt`
`&&` / `||`	Both conditions / either condition
`? :`	Ternary choice: `$count === 1 ? 'product' : 'products'`
`??`	Missing/null fallback: `$name ?? 'Unknown'`
`if` / `return`	Choose a branch / end the call and return its result
`function`	Declare a function/method or anonymous closure
`use ($request)`	Capture an outer variable inside a closure
`namespace` / top-level `use`	Organize and import class names
`class` / `new`	Define a class / construct an object
`extends`	Inherit a parent class, such as Product extending Model
`implements`	Declare an interface contract, needed for enforced MustVerifyEmail

Expression	Meaning and example
trait `use`	Include reusable behavior such as Notifiable inside a class
`public` / `protected` / `private`	External / class-and-subclass / declaring-class access
`$this`	Current instance inside a method
`: View` / `: void` / `A|B`	Return type / no return value / allowed union types
named argument	`absolute: false` selects an argument by parameter name
`throw`	Raise an exception and interrupt normal execution
`match`	Strict expression selection with a default arm; used for component sizing/alignment
`__DIR__`	Current source file directory, independent of terminal folder
`require` / `require_once`	Load and execute another PHP file / ensure it is loaded once
`//` / `/* */` / `/** */`	Line comment / block comment / tooling documentation comment

`match` example from the component idea:

```
$widthClass = match ($width) {
    '48' => 'w-48',
    default => $width,
};
```

The expression compares strictly, chooses one result, and has no switch-style fallthrough. `default` handles inputs not listed. Component props should be understood before changing their allowed values.

Laravel helpers and request/response methods

Call	Purpose in the application
`view(name, data)`	Render a Blade view with supplied variables
`compact('products')`	PHP helper making an array from named local variables
`route(name, parameters)`	Generate a named route URL, including product IDs
`url('/')`	Generate the application's root URL
`asset('css/style.css')`	Generate an asset URL under public/
`redirect()`	Build a redirect response
`redirect()->route(...)`	Redirect to a named route
`redirect()->intended(fallback)`	Return to saved intended destination, otherwise fallback
`back()`	Redirect to the previous location
`->with('status', value)`	Flash feedback to the next request/session

Call	Purpose in the application
`->withInput(...)`	Flash selected old form input for the next view
`->withErrors(...)`	Place validation/broker messages into error bags
`old(field, fallback)`	Read previous input; use fallback if absent
`session('status')`	Read a session value such as temporary feedback
`auth()->check()`	Check authenticated state for the current guard
`$request->user()`	Current authenticated User, not a submitted user ID
`$request->validate(rules)`	Check rules or throw; returns validated values on success
`validateWithBag(name, rules)`	Validate using a named error bag for a specific form
`$request->validated()`	Read accepted fields from a resolved FormRequest
`$request->only(keys)`	Select keys; by itself it does not validate their values
`$request->boolean('remember')`	Interpret a submitted checkbox as a boolean
`$request->route('token')`	Read a matched route parameter
`$request->ip()`	Request IP used as part of the login limiter key
`$request->session()`	Access session operations
`session()->regenerate()`	Rotate session ID after successful login
`session()->invalidate()`	Discard session data and regenerate its ID
`session()->regenerateToken()`	Rotate the CSRF token
`session()->put(key, value)`	Persist a session value such as confirmation time
`config(key)`	Read resolved Laravel configuration
`env(key, default)`	Read environment values, mainly in config definitions
`__()`	Translate a key/text into a display message
`event(new Event(...))`	Dispatch an event to registered listeners
`number_format(value, 2)`	PHP display formatting for two-decimal prices
`dirname(path)`	PHP path parent helper for startup file locations
`file_exists(path)`	Check whether a startup/maintenance file exists
`microtime(true)`	Float timestamp used to record Laravel startup time
`time()`	Current Unix timestamp for password-confirmation session state

Authentication and model methods

Call/setting	Purpose and boundary
`Auth::attempt(credentials, remember)`	Check a saved account/password hash and authenticate if valid
`Auth::login($user)`	Sign in an already-selected User, used after registration
`Auth::guard('web')`	Select the browser session guard
`guard()->validate(credentials)`	Check credentials without creating a new login session
`logout()`	End authenticated state; session invalidation follows separately
`Hash::make(password)`	Produce a password hash for storage
`Password::sendResetLink(...)`	Broker creates/sends a reset request, not a new password
`Password::reset(data, callback)`	Verify reset credentials/token, then invoke the password-change callback
`Rules\Password::defaults()`	Configured Password rule object; default minimum is eight without customization
`Str::random(60)`	New random remember token when resetting a password
`Str::lower` / `transliterate`	Normalize login limiter key text
`RateLimiter::hit` / `clear`	Record a failed attempt / remove its counter after success
`tooManyAttempts` / `availableIn`	Check the limit / remaining wait seconds
`ValidationException::withMessages`	Raise field-specific errors for Laravel to handle
`Product::all()`	Read a collection of all product rows
`Model::create(data)`	Insert a model using allowed mass-assignment keys
`$model->fill(data)`	Assign allowed attributes in memory; saving is a separate call
`$model->update(data)`	Assign and save changes to that existing record
`$model->save()`	Persist the model's current attribute state
`$model->delete()`	Delete this record; no Product soft deletes are configured
`$model->forceFill(data)`	Assign despite fillable; use only explicit trusted keys
`isDirty('email')`	Detect an unsaved change from the originally loaded value
`$collection->count()`	Number of loaded values/records
`$fillable`	Keys allowed in mass assignment
`$hidden`	Attributes omitted when serializing; does not alter DB storage
`casts()`	Attribute conversions, including User dates/password hashes

Call/setting	Purpose and boundary
`hasVerifiedEmail` / `markEmailAsVerified`	Inspect/set verification status; routes alone do not enforce it
`sendEmailVerificationNotification()`	Dispatch verification email, separate from password reset

Schema and validation terms

Name	Meaning
`Schema::create`	Create table structure described by Blueprint
`$table->id()`	Auto-incrementing primary key
`string` / `text` / `integer`	Short text / longer text / whole-number column
`decimal(8, 2)`	Eight total digits, two decimal places
`nullable()`	SQL NULL allowed
`unique()` / `primary()`	Unique constraint / primary key
`index()`	Database index supporting lookups; not automatically an authorization rule
`timestamps()`	created_at and updated_at columns
`rememberToken()`	User remember_token column
`foreignId()`	ID-shaped column; a foreign key still needs a constraint definition
`dropIfExists()`	Drop an existing table during rollback
`required` / `nullable`	Value required / null allowed
`string` / `integer` / `numeric` rules	Validate value type/representation
`min` / `max`	Bounds interpreted with the other rules/type
`decimal:0,2`	Zero through two decimal places accepted
`email` / `lowercase` / `unique`	Email shape / lowercase / unused saved email
`confirmed`	Match the corresponding _confirmation field
`current_password`	Verify the authenticated user's current password

Blade and generated component expressions

Expression	Purpose
`@extends` / `@section` / `@yield`	Inheritance layout, provided section, insertion point
`x-guest-layout` / `x-app-layout`	Component layouts with inner markup as slot

Expression	Purpose
`$slot` / named `x-slot`	Default/named component content
`@include`	Insert a reusable view partial
`{{ ... }}`	Escaped output
`@php`	Embedded PHP; our comment-only blocks produce no visible text
`@if` / `@isset` / `@auth`	Condition / existence / authentication-dependent blocks
`@foreach` / `@forelse` / `@empty`	Loops with optional empty behavior
`@error` / `$message`	Selected field error and its text
`@csrf` / `@method`	Hidden session token / spoofed PUT-DELETE form method
`@checked`	Render checked attribute when its condition is true
`@props`	Anonymous component input/default declarations
`$attributes->merge`	Combine caller attributes with component defaults
`$attributes->has`	Check whether caller supplied an attribute
`@js`	Encode server data safely for a JavaScript context
`@vite`	Link Vite's development or compiled entry assets
`$errors->get/any/all`	Field messages / whether errors exist / all messages
named error bag	Separate profile deletion/password errors from other forms

JavaScript and browser methods

Expression	Purpose and example
`const` / `let`	Stable variable binding / reassigned binding
`document.getElementById`	Find theme button by ID
`querySelector` / `querySelectorAll`	First match or null / matching NodeList
`forEach(function (...) {...})`	Process every matched password input
`parentElement`	Password input's wrapper element
`classList.add/toggle/contains`	Add/select/check CSS classes
`createElement('button')`	Build a DOM button without submitting a form
`setAttribute`	Set accessible labels, pressed state, or controlled input ID
`textContent`	Display a string as text, such as Show/Hide

Expression	Purpose and example
`appendChild`	Add the Show/Hide button to the wrapper
`addEventListener('click', callback)`	Run code when that control is clicked
`input.type`	Switch browser presentation between password and text
`trim()` / `toLowerCase()`	Remove edge whitespace / normalize label case
`String(boolean)`	Convert accessible pressed value to true/false text
`localStorage.getItem/setItem`	Read/store theme preference in this browser
`try` / `catch`	Handle blocked browser storage without breaking the toggle
`setTimeout(callback, 5000)`	Schedule banner removal after five seconds
`remove()`	Remove the selected DOM banner, not a DB row
`confirm()`	Browser OK/Cancel prompt returning a boolean
Alpine `x-data/x-show/x-on`	UI state, visibility, and events in generated components
Alpine `$dispatch/$watch/$el`	Custom events, state observation, component element

CSS/build/test terms

Expression	Purpose
`rgba(...)`	Color with alpha transparency
`linear-gradient/radial-gradient`	Directional or elliptical backdrop/color variation
`backdrop-filter`	Blur/saturate the surface behind the element
`box-shadow: inset ...`	Highlight/shadow inside the surface
`display:flex` / `gap` / `flex-wrap`	Child alignment, spacing, wrapping
`position:relative/absolute`	Wrapper context and positioned icon/control
`transform:translateY(-50%)`	Center an icon relative to its own height
`white-space:nowrap`	Keep a button label together
`:focus-visible` / `:hover`	Keyboard-focus/pointer states
`@media`	Conditional rules for size/motion preference
`font:inherit`	Controls inherit the surrounding font settings
`npm run build`	Vite builds the configured resource assets
`Notification::fake()`	Capture test notifications without sending real mail

Expression	Purpose
`User::factory()->create()`	Generate a test User record
`RefreshDatabase`	Reset configured test DB between relevant tests
`actingAs($user)`	Authenticate a test request as a fixture user
`get/post/put/delete` test methods	Simulate requests with specified HTTP methods
`assertOk/assertRedirect`	Verify response success/destination
`assertAuthenticated/assertGuest`	Verify authentication state
`assertSessionHasErrors`	Verify rejected input produces the named errors
`assertDatabaseCount/Has`	Check number/content of test records
`assertCookie`	Verify remembered login issued the expected cookie

Find the real calling file in CODE_MAP.md, then use the complete guide for the surrounding request flow. A method name alone does not tell you whether it changes the database, changes only an object in memory, or updates browser markup.

Application file map

Paths start inside `app-crud/`. Open a file from the links below; its header comments name the calling/related files.

Workspace around the application

Location	Purpose
app-crud/	Laravel source; run Artisan/Composer/npm here
docs/START_HERE.md	Current learning-pack entry point
docs/TERMS_AND_CONCEPTS.md	Definitions, reasons, project examples, and concept checks
docs/COMPLETE_STUDY_GUIDE.md	Language concepts and request-flow explanations
docs/LAB3_ANSWERS.md	Eight course questions and required evidence
docs/PRACTICE_AND_DEMO.md	Prediction exercises and demonstration plan
docs/VERIFICATION.md	Actual checks and remaining manual verification
LAB2_STUDY_GUIDE.md and output/pdf/LAB2_STUDY_GUIDE.pdf	Earlier Lab 2 checkpoint documentation
PRODUCT.md	Confirmed project scope
.git/	Local version history; do not edit internal files
.agents/, .codex/, .impeccable/	Local agent/design tooling, excluded from Git
tmp/ and .codex-pdf-tools/	Preparation/check/render tools, excluded from Git

Active source and generated scaffold

Read entry points and routes first, then controllers/requests/models, then views and assets. The alphabetical groups below let you locate a specific file afterward.

Application root

.editorconfig

Open source

Editor spacing, line endings, and charset preferences. Applies to editor formatting, not controller runtime behavior.

.env.example

Open source

Shareable environment template with empty/default values. Copy locally to .env and supply private values. It is not the current secret configuration.

.gitattributes

Open source

Git text/diff settings for the Laravel source. Line-ending rules explain CRLF/LF conversion messages when saving on Windows.

.gitignore

Open source

Laravel excludes env/dependency/runtime artifacts. The workspace root ignore file additionally excludes agent/PDF preparation folders and local database files.

README.md

Open source

Links to the current learning pack and workspace setup instructions, followed by the retained generated Laravel framework README.

artisan

Open source

Terminal entry point. PHP loads the same application bootstrap used by web requests, but handles an Artisan command such as serve, migrate, or route:list instead of returning a browser page.

composer.json

Open source

PHP package requirements and Composer scripts. The require sections choose Laravel/Breeze/tools; scripts run setup/test/discovery steps. JSON cannot contain comments.

composer.lock

Open source

Exact PHP dependency versions resolved by Composer. Commit it for repeatable installation; do not annotate or manually edit its package records.

package-lock.json

Open source

Exact resolved Node dependency versions used by npm. Generated by npm and kept in Git; not a source file for manual teaching comments.

package.json

Open source

Node package requirements and npm scripts. build runs Vite build and dev runs the Vite development server. JSON cannot contain comments.

phpunit.xml

Open source

PHPUnit suites/source scope and testing environment. SQLite :memory:, array sessions/cache/mail, and lower test hashing cost isolate tests from real database/email.

postcss.config.js

Open source

Build pipeline loads Tailwind and Autoprefixer for generated CSS. This file is read by tooling, not by the browser as a script.

tailwind.config.js

Open source

Tailwind scans Blade files for utility classes, includes its forms plugin, and defines scaffold fonts. The custom plain stylesheet separately selects Inter on auth/product pages.

vite.config.js

Open source

Vite registers CSS/JavaScript entry files with the Laravel plugin and refreshes development pages. npm run build creates public/build/manifest.json used by the vite directive.

app

app/Http/Controllers/Auth/AuthenticatedSessionController.php

Open source

Login/logout controller called from routes/auth.php. LoginRequest validates credentials; successful login regenerates the session and returns to the intended page or Products. Logout clears the session and returns to Login.

Methods to locate: `create()`, `store()`, `destroy()`.

app/Http/Controllers/Auth/ConfirmablePasswordController.php

Open source

Password confirmation for a signed-in user. Verifies the current password and records confirmation time in the session. It does not change the password or automatically protect every product action.

Methods to locate: `show()`, `store()`.

app/Http/Controllers/Auth/EmailVerificationNotificationController.php

Open source

Resends a verification notification for the signed-in user. routes/auth.php limits these requests. This generated feature is separate from the tested password-reset email flow.

Methods to locate: `store()`.

app/Http/Controllers/Auth/EmailVerificationPromptController.php

Open source

Invokable controller for the verification notice. Shows auth.verify-email when needed. Routes exist, but full verification enforcement requires User to implement MustVerifyEmail.

Methods to locate: `__invoke()`.

app/Http/Controllers/Auth/NewPasswordController.php

Open source

Reset Password controller. The email link supplies a token and email to the view. The POST handler validates input, asks the password broker to verify the token, hashes the new password, and returns to Login with feedback.

Methods to locate: `create()`, `store()`.

app/Http/Controllers/Auth/PasswordController.php

Open source

Authenticated profile-password update. The profile partial submits current_password and the new password here. This flow requires the current password and does not use an emailed reset token.

Methods to locate: `update()`.

app/Http/Controllers/Auth/PasswordResetLinkController.php

Open source

Forgot Password controller for password.request/password.email routes. The view collects an email; the password broker creates a reset token and sends a notification through config/mail.php. No password is changed at this stage.

Methods to locate: `create()`, `store()`.

app/Http/Controllers/Auth/RegisteredUserController.php

Open source

Registration controller for GET/POST register. Validates account details, creates a User with a password hash, fires Registered, signs in that user, then redirects to Dashboard. Registration currently has a different default destination from login.

Methods to locate: `create()`, `store()`.

app/Http/Controllers/Auth/VerifyEmailController.php

Open source

Handles signed verification URLs, marks the signed-in user email verified, and fires Verified. Laravel validates the request signature and route identity before this controller runs.

Methods to locate: `__invoke()`.

app/Http/Controllers/Controller.php

Open source

Shared parent of the application controllers. Concrete controllers contain the request methods; this empty abstract class keeps a common place for future shared behavior.

app/Http/Controllers/ProductController.php

Open source

CRUD coordinator called by the six protected product routes in routes/web.php. Reads and validates input, calls the Product model, then returns a products view or a redirect with success feedback.

Methods to locate: `index()`, `create()`, `store()`, `edit()`, `update()`, `destroy()`.

app/Http/Controllers/ProfileController.php

Open source

Generated Breeze profile actions: display the current user, update profile details, or delete the account after current-password validation. Product records have no user_id relationship, so deleting an account does not cascade-delete products.

Methods to locate: `edit()`, `update()`, `destroy()`.

app/Http/Requests/Auth/LoginRequest.php

Open source

Login-specific validation object injected into AuthenticatedSessionController::store(). Checks input, limits failed login attempts, and asks the web guard to authenticate using the remember checkbox value.

Methods to locate: `authorize()`, `rules()`, `authenticate()`, `ensureIsNotRateLimited()`, `throttleKey()`.

app/Http/Requests/ProfileUpdateRequest.php

Open source

Rules for profile name/email changes. The unique-email rule ignores the current user ID so keeping the same email is allowed while taking another user email is rejected.

Methods to locate: `rules()`.

app/Models/Product.php

Open source

Product model for the products table. ProductController uses all(), create(), update(), and delete() inherited from Eloquent. The products migration defines columns; this model works with the rows.

app/Models/User.php

Open source

User model for the users table. Authentication uses this class to load accounts. HasFactory creates test users; Notifiable sends reset/verification notifications. Email verification is not enforced because this class does not implement MustVerifyEmail.

Methods to locate: `casts()`.

app/Providers/AppServiceProvider.php

Open source

Startup extension point registered in bootstrap/providers.php. register() can bind services; boot() runs after registration. Both are empty here because the labs use Laravel defaults.

Methods to locate: `register()`, `boot()`.

app/View/Components/AppLayout.php

Open source

Class component behind x-app-layout on Dashboard/Profile. render() selects layouts.app, where component content is supplied as $slot.

Methods to locate: `render()`.

app/View/Components/GuestLayout.php

Open source

Class component behind x-guest-layout. render() selects layouts.guest, which wraps Login/Register/Forgot/Reset pages and loads the custom auth styles.

Methods to locate: `render()`.

bootstrap

bootstrap/app.php

Open source

Application setup. Registers routes/web.php, routes/console.php, and the health endpoint. Laravel supplies its default middleware; named auth/guest middleware is applied in the route files.

bootstrap/providers.php

Open source

Lists application service providers loaded during startup. AppServiceProvider supplies application-wide registration and boot hooks.

config

config/app.php

Open source

Application name, URL, locale, key, timezone, and general settings. env() reads deployment values; never copy the real key into a public explanation.

config/auth.php

Open source

Session web guard, Eloquent User provider, and password broker settings. Reset expiry is in minutes; throttle is in seconds. Role/admin checks are not defined here.

config/cache.php

Open source

Cache stores and prefix. Laravel rate limiting can use the configured store; clearing cache is different from deleting product records.

config/database.php

Open source

Database connections and migration/cache configuration. .env selects the mysql connection to XAMPP MariaDB; tests use isolated in-memory SQLite.

config/filesystems.php

Open source

Storage disks, paths, and public-storage link definitions. Does not define product database columns or permissions.

config/logging.php

Open source

Log channels and destinations for application events/errors. Runtime logs may include private details and are excluded from Git.

config/mail.php

Open source

Mailer definitions and sender defaults. smtp uses MAIL_SCHEME/HOST/PORT/USERNAME/PASSWORD. The password broker notification reaches this mailer; keep real credentials in .env only.

config/queue.php

Open source

Queue connections and failed-job settings. Tables exist for optional background jobs; a queue worker is only needed when work is explicitly queued.

config/services.php

Open source

Optional third-party integration settings. The current reset email uses SMTP config rather than these optional service transports.

config/session.php

Open source

Session driver, lifetime, cookie, and related options. Database sessions connect browser requests to authenticated users through the sessions table.

database

database/factories/UserFactory.php

Open source

Creates fake User data for tests. Factory users are test fixtures; fake values do not register real users or send real email during the test suite.

Methods to locate: `definition()`, `unverified()`.

database/migrations/0001_01_01_000000_create_users_table.php

Open source

Creates users, password_reset_tokens, and sessions. User/auth controllers use users; the password broker uses reset tokens; the configured database session driver uses sessions.

Methods to locate: `up()`, `down()`.

database/migrations/0001_01_01_000001_create_cache_table.php

Open source

Creates database cache/lock tables. Laravel can use these for cached values and rate limiting when the configured cache store is database.

Methods to locate: `up()`, `down()`.

database/migrations/0001_01_01_000002_create_jobs_table.php

Open source

Creates queued-job, batch, and failed-job tables. These support background jobs; our reset notification is sent synchronously in the current flow.

Methods to locate: `up()`, `down()`.

database/migrations/2026_10_07_015613_create_products_table.php

Open source

Creates products for the Product model. up() defines columns; down() drops the whole table on rollback. A migration changes structure; it does not submit product form values.

Methods to locate: `up()`, `down()`.

database/seeders/DatabaseSeeder.php

Open source

Optional sample-data entry point for db:seed. Do not run it just to view the application; normal registration and product forms create records through controllers.

Methods to locate: `run()`.

public

public/.htaccess

Open source

Apache rewrite rules that direct application URLs to public/index.php. Artisan serve uses its own development routing, so this Apache file is not the primary server in our tests.

public/css/auth.css

Open source

Authentication-specific styles loaded after style.css. Centers guest forms, handles field icons/Show-Hide controls, checkbox layout, errors, and contrasting primary buttons in both themes.

public/css/style.css

Open source

Shared product/glass theme styling. Product layout and guest layout load this URL directly. Controls Inter, backgrounds, buttons, table/forms, icons, keyboard focus, and the light-theme overrides.

public/favicon.ico

Open source

Browser-tab icon asset. Binary files have no comment syntax; this retained scaffold asset does not implement authentication.

public/index.php

Open source

HTTP entry point. The browser request reaches this file, which loads Composer classes and bootstrap/app.php, then gives the captured request to Laravel. Routes select the next controller; this file does not perform CRUD itself.

public/js/password-visibility.js

Open source

Adds a separate Show/Hide button to each styled password input. Changes the input type in the browser without changing its value, field name, authentication, or hashing.

public/js/product-messages.js

Open source

Removes only the product success banner after five seconds. The controller already completed the database operation before the list page and this timer appear.

public/js/theme.js

Open source

Appearance-only code loaded with defer by product/guest layouts. A click toggles light-theme on body and saves the browser choice in localStorage; it does not change account permissions.

public/robots.txt

Open source

Crawler guidance only. It is not login protection and does not prevent access to a known URL; auth middleware protects product routes.

resources

resources/css/app.css

Open source

Tailwind source compiled by Vite. Dashboard/Profile use this build; the guest layout loads it as well, followed by custom styles that control the four redesigned forms.

resources/js/app.js

Open source

Vite entry for generated Breeze pages. Loads bootstrap.js, exposes Alpine, and starts Alpine for dropdowns/modals in Dashboard/Profile. Custom auth/product scripts are linked separately from public/js.

resources/js/bootstrap.js

Open source

Initializes Axios and its XMLHttpRequest header for JavaScript HTTP requests. Our normal HTML CRUD forms do not need Axios; this remains Breeze scaffold.

resources/views/auth/confirm-password.blade.php

Open source

Generated current-password confirmation form for an authenticated session. This is distinct from email-based password recovery.

resources/views/auth/forgot-password.blade.php

Open source

Reset-link request form posts to password.email. This sends an email without changing the password; status/errors show broker results and throttling.

resources/views/auth/login.blade.php

Open source

Login form posts to login. email/password/remember names match LoginRequest; CSRF protects the form. Errors and reset-status messages are rendered inside x-guest-layout.

resources/views/auth/register.blade.php

Open source

Registration form posts to register. Name/email/password/password_confirmation match RegisteredUserController validation. Password fields never use old().

resources/views/auth/reset-password.blade.php

Open source

New-password form posts to password.store. Preserve the hidden token from the reset URL and both password field names; NewPasswordController verifies and saves the change.

resources/views/auth/verify-email.blade.php

Open source

Generated notice/resend/logout view for optional email verification. It does not prove verification is required by this app; User currently lacks the enforcing interface.

resources/views/components/application-logo.blade.php

Open source

Generated Laravel SVG logo for Dashboard/Profile scaffold. Custom product/guest navbars draw a separate package icon.

resources/views/components/auth-session-status.blade.php

Open source

Generated component for session feedback used by scaffold auth views. The four customized auth views display status directly instead.

resources/views/components/danger-button.blade.php

Open source

Generated destructive-action button in profile account deletion. Product deletion has its own button and form in products.index.

resources/views/components/dropdown-link.blade.php

Open source

Generated link component for account menu destinations. The logout menu triggers a separate POST form instead of changing state through a GET URL.

resources/views/components/dropdown.blade.php

Open source

Reusable anonymous Blade component for generated navigation. Props choose alignment/width; named trigger/content slots carry markup. Alpine tracks open state and click-outside behavior.

resources/views/components/input-error.blade.php

Open source

Reusable error-list component in generated profile/confirmation views. The four custom auth pages instead use error directives and auth-error styling.

resources/views/components/input-label.blade.php

Open source

Reusable label component: receives a value and forwards HTML attributes. Native labels are used in the four custom auth forms.

resources/views/components/modal.blade.php

Open source

Reusable Alpine modal for profile deletion. Props set name/width; events open/close it; focus helpers move keyboard focus and body scrolling is locked while visible.

resources/views/components/nav-link.blade.php

Open source

Generated navigation link with active-state classes. Dashboard navigation uses it; the product navbar does not.

resources/views/components/primary-button.blade.php

Open source

Generated submit-button component for Breeze scaffold pages. Custom auth forms use auth-submit; product forms use button-primary.

resources/views/components/responsive-nav-link.blade.php

Open source

Mobile variant of the generated active navigation link for layouts.navigation.

resources/views/components/secondary-button.blade.php

Open source

Generated non-submit button component used for actions such as cancelling the profile modal. Default type prevents unintended form submission.

resources/views/components/text-input.blade.php

Open source

Reusable scaffold input. Merges attributes and supports disabled; redesigned auth fields use normal HTML inputs to keep their custom styling simple.

resources/views/dashboard.blade.php

Open source

Generated logged-in dashboard using x-app-layout. Registration currently redirects here. Login defaults to Products, so these destinations are intentionally documented separately.

resources/views/layouts/app.blade.php

Open source

Generated component layout for Dashboard/Profile via x-app-layout. Includes layouts.navigation and renders $header/$slot. Product pages must extend layouts.product instead.

resources/views/layouts/guest.blade.php

Open source

Component layout for x-guest-layout. Receives $slot from authentication views; loads Inter, Vite assets, custom CSS, theme.js, and password-visibility.js.

resources/views/layouts/navigation.blade.php

Open source

Generated Breeze navigation for Dashboard/Profile. Alpine handles menus; its logout form posts to logout. The product navbar is separate in layouts.product.

resources/views/layouts/product.blade.php

Open source

Section-based layout for products views. Uses title/content yields, not $slot. Loads custom CSS, theme.js, product-messages.js, and includes a POST logout form.

resources/views/products/create.blade.php

Open source

Product form that posts to product.store. Field names match controller validation. old() restores failed submissions; errors remain visible. Extends layouts.product.

resources/views/products/edit.blade.php

Open source

Product form that posts with a hidden PUT method to product.update. Route binding supplies the selected record; old(field, savedValue) chooses submitted or stored data.

resources/views/products/index.blade.php

Open source

ProductController::index passes $products here. Render rows, empty state, edit links, and CSRF-protected DELETE forms. The success banner is removed later by product-messages.js.

resources/views/profile/edit.blade.php

Open source

Profile view receives the authenticated User from ProfileController. Includes separate information, password, and account-deletion partials using the generated app layout.

resources/views/profile/partials/delete-user-form.blade.php

Open source

Generated account-deletion modal using x-modal and Alpine. Posts with DELETE method to profile.destroy and requires the current password. Product ownership/cascade deletion is not configured.

resources/views/profile/partials/update-password-form.blade.php

Open source

Submits PUT password.update with current_password/new password/confirmation. A named error bag keeps these errors separate from other profile forms.

resources/views/profile/partials/update-profile-information-form.blade.php

Open source

Submits PATCH profile.update with name/email. Displays saved feedback and optional verification controls; ProfileUpdateRequest validates before ProfileController saves.

resources/views/welcome.blade.php

Open source

Original Laravel welcome template retained as scaffold. The current home route redirects to Login/Products instead, so it does not render this page.

routes

routes/auth.php

Open source

Breeze URL map. guest protects registration/login/password-recovery pages from signed-in users. auth protects logout/profile-password/verification routes. GET displays forms; POST/PUT processes changes.

routes/console.php

Open source

Artisan command routes, not browser routes. The sample inspire command is generated scaffold and does not participate in the product or login flows.

routes/web.php

Open source

Browser URL map. Home redirects based on login state. Product/Profile groups require auth. The last require loads routes/auth.php once. Product routes use destroy(), matching ProductController.

tests

tests/Feature/Auth/AuthenticationTest.php

Open source

Exercises login display, valid/invalid credentials, and logout against a temporary database. Redirect expectations must follow the customized Product/Login destinations.

Methods to locate: `test_login_screen_can_be_rendered()`, `test_users_can_authenticate_using_the_login_screen()`, `test_users_can_not_authenticate_with_invalid_password()`, `test_users_can_logout()`.

tests/Feature/Auth/EmailVerificationTest.php

Open source

Tests generated verification handlers. Passing does not make verification mandatory; User must implement MustVerifyEmail for enforcement.

Methods to locate: `test_email_verification_screen_can_be_rendered()`, `test_email_can_be_verified()`, `test_email_is_not_verified_with_invalid_hash()`.

tests/Feature/Auth/PasswordConfirmationTest.php

Open source

Tests confirmation of the current password for authenticated users. Does not replace login validation or the reset-link workflow.

Methods to locate: `test_confirm_password_screen_can_be_rendered()`, `test_password_can_be_confirmed()`, `test_password_is_not_confirmed_with_invalid_password()`.

tests/Feature/Auth/PasswordResetTest.php

Open source

Exercises reset notification and token-based password changes. Notification::fake captures notifications in memory, so these tests do not send mail through Gmail.

Methods to locate: `test_reset_password_link_screen_can_be_rendered()`, `test_reset_password_link_can_be_requested()`, `test_reset_password_screen_can_be_rendered()`, `test_password_can_be_reset_with_valid_token()`.

tests/Feature/Auth/PasswordUpdateTest.php

Open source

Tests authenticated profile-password updates with the current password. Uses generated temporary users and checks stored hashes.

Methods to locate: `test_password_can_be_updated()`, `test_correct_password_must_be_provided_to_update_password()`.

tests/Feature/Auth/RegistrationTest.php

Open source

Exercises registration display and valid account creation. Registration still redirects to Dashboard; new validation checks reject duplicate email and mismatched passwords.

Methods to locate: `test_registration_screen_can_be_rendered()`, `test_new_users_can_register()`.

tests/Feature/ExampleTest.php

Open source

Checks the home route response. Home now redirects unauthenticated visitors to login, so the expected response is a redirect rather than an HTML 200.

Methods to locate: `test_the_application_returns_a_successful_response()`.

tests/Feature/LabThreeFlowTest.php

Open source

Tests signed-in home redirect, all six guest-protected product routes, intended login redirect, registration rejection, remember cookie, and the product logout form using isolated in-memory data.

Methods to locate: `test_home_sends_authenticated_users_to_products()`, `test_all_six_product_endpoints_require_login()`, `test_login_returns_to_the_original_protected_page()`, `test_duplicate_email_and_mismatched_password_are_rejected()`, `test_remembered_login_issues_a_recaller_cookie()`, `test_authenticated_product_view_contains_logout_form()`.

tests/Feature/ProfileTest.php

Open source

Tests profile display, edits, and account deletion in the temporary database. The scaffold profile UI remains separate from the four customized auth pages.

Methods to locate: `test_profile_page_is_displayed()`, `test_profile_information_can_be_updated()`, `test_email_verification_status_is_unchanged_when_the_email_address_is_unchanged()`, `test_user_can_delete_their_account()`, `test_correct_password_must_be_provided_to_delete_account()`.

tests/TestCase.php

Open source

Base Laravel test class that boots the app. Feature tests extend it; database-backed tests add RefreshDatabase and use phpunit.xml testing settings.

tests/Unit/ExampleTest.php

Open source

Generated simple assertion example. A unit test checks a small piece of behavior; this sample does not exercise HTTP routes or a real database.

Methods to locate: `test_that_true_is_true()`.

Excluded runtime and third-party folders

Location	Meaning
vendor/	Composer-installed Laravel/Symfony/PHPUnit and other package code
node_modules/	npm-installed frontend/build packages
public/build/	Vite compiled CSS/JS and manifest; regenerate using npm run build
bootstrap/cache/	Generated framework cache files; keep the provided ignore file
storage/framework/views/	Compiled Blade PHP; edit the original Blade source instead

Location	Meaning
storage/framework/sessions/	File-session runtime location when that driver is selected
storage/logs/	Runtime logs, potentially containing private details
storage/app/	Application file storage; not the MySQL product table

The root .env is not versioned. Runtime folders retain ignore files so required directory structure can exist after cloning.

Reading one file without getting lost

Find its route/caller, read the method signature, follow its input checks, identify any data-changing call, and identify the returned response. For a view, identify its layout, required variables, form action/method, field names, and feedback blocks. For a script, identify its selector and event/timer. Use the complete guide for syntax explanations.

Verification of the documented application

Checked October 13, 2026, in the current workspace. This report separates source checks, server behavior tests, and user demonstrations.

Source and documentation changes

- Explanatory notes cover 97 application files, including entry points, routes, controllers, request classes, models, views/components, configuration/build files, database schema/factories, and tests.
- The file map lists the application source plus package manifests/lockfiles, editor/Git settings, and runtime/third-party folders. JSON and binary contents are explained without adding invalid comments.
- Source comparisons remove PHP/Blade/CSS/JavaScript comments before comparing executable content. All 97 inspected files matched the starting source except the explicitly allowed generated-test redirect expectation changes.
- The auth templates' pasted icon-wrapper/comment indentation was cleaned without changing field values/names, CSRF/token expressions, or controller actions.
- The actual private .env, SMTP values, MySQL records, and third-party dependencies were not rewritten. No Git push was performed as part of this documentation task.

Automated checks

The application tests passed: 31 tests, 95 assertions. This includes login/logout, registration, reset-token flows, profile behavior, and six focused Lab 3 checks: authenticated home redirect, all six guest-protected product endpoints, intended login redirect, duplicate-email/password-confirmation rejection, remember-cookie issuance, and the product logout form.

phpunit.xml selects SQLite :memory: and array mail/session/cache for testing. Notification fakes prevent reset tests from sending actual Gmail messages. All application PHP files passed syntax checks, the three custom public scripts passed JavaScript syntax checks, and Blade compilation passed. The Vite production build completed successfully after rerunning outside a directory-restricted sandbox. Product tests verify rejected guest write requests leave test records unchanged. The test suite does not prove final visual appearance or long-term browser-cookie behavior.

User demonstrations in the conversation

The student demonstrated account registration, login, logout, login rejection, guest redirects, authenticated CRUD, and resetting passwords through both Mailtrap and Gmail delivery. Both providers were tested sequentially. Desktop screenshots show the customized forms, theme behavior, and input icon alignment.

Still capture/check before submission

- Registration error screenshot and the successfully registered-user evidence.
- Final Reset Password screenshots with URL/token hidden.
- Narrow-screen checks of all updated auth/product pages, including the new logout tools and Show/Hide controls.
- Long-term Remember me behavior in the chosen browser, with awareness that cookies and explicit logout affect persistence.

- The eight required deliverables listed in LAB3_ANSWERS.md, matched to instructor format.
- A reviewed Lab 3 commit/push once source and evidence are ready. Lab 2's tag remains intact.

Feature boundaries to explain accurately

Registration still opens Dashboard; normal login defaults to Products. User does not implement MustVerifyEmail, so generated verification routes do not enforce verification for product access. All authenticated users share the product list; there is no owner relationship, administrator role, or product authorization policy. Dashboard/Profile keep generated Breeze styling rather than the customized guest layout.

Instructor questions: files, lines, and code to show

Line numbers were captured from the saved source on October 13, 2026. They are a navigation aid, not something to memorize. Edits can move them; use the method or code expression below as the reliable search anchor. All application paths start inside app-crud unless labelled workspace root.

How to locate the answer during a demonstration

1. In VS Code press Ctrl+P and enter the listed file path.
2. Press Ctrl+G, enter the start line number, and press Enter.
3. Point to the expression, name what it checks/does, and explain the next connected file.
4. Use Ctrl+F for the method/expression if your line numbers have changed.

The numbered excerpts are reading aids. Do not paste them into PHP because the line numbers are not valid source code. Never show the private .env, reset URL, cookies, or SMTP password while answering.

Quick question-to-file index

Question	First file to open	First code to point to
1	composer.json	Line 15: Installed Breeze dependency
2	app/Http/Controllers/Auth/RegisteredUserController.php	Line 41: Registration validates account details
3	routes/web.php	Line 42: Product routes: auth applied to the whole group
4	routes/web.php	Line 42: All product display/write routes protected
5	config/mail.php	Line 9: Default mailer selected from MAIL_MAILER
6	config/mail.php	Line 21: Private SMTP username/password are read, not hard-coded
7	app/Http/Requests/Auth/LoginRequest.php	Line 56: Authentication: credential check
8	routes/web.php	Line 42: Product group uses auth only

Question 1: What is Laravel Breeze, and what problem does it solve?

Show the installed package first, then one generated route/controller/view connection. Breeze supplies account scaffolding; the package declaration alone does not prove every feature has been configured.

Say this while pointing:

"Breeze is installed as a development package. Its auth routes call these controllers and views for registration, login, logout, and recovery. We customized those views and added auth protection to our product routes separately."

Installed Breeze dependency

File: `composer.json`. Line 15.

```
15:         "laravel/breeze": "^2.4",
```

Login route selects AuthenticatedSessionController::create

File: `routes/auth.php`. Line 27 through 28.

```
27:     Route::get('login', [AuthenticatedSessionController::class, 'create'])
28:         ->name('login');
```

Controller returns the login view

File: `app/Http/Controllers/Auth/AuthenticatedSessionController.php`. Line 22 through 25.

```
22:     public function create(): View
23:     {
24:         return view('auth.login');
25:     }
```

Login form posts to Breeze login route

File: `resources/views/auth/login.blade.php`. Line 23.

```
23:     <form method="POST" action="{{ route('login') }}">
```

Question 2: What is the difference between registration and authentication?

Show account insertion in registration, then credential checking in LoginRequest. They are separate operations even though registration subsequently signs in the new account.

Say this while pointing:

"Registration creates a User record with a hashed password. Authentication checks the saved credentials with Auth::attempt and establishes the signed-in session; it does not insert another user."

Registration validates account details

File: `app/Http/Controllers/Auth/RegisteredUserController.php`. Line 41 through 46.

```
41:         $request->validate([
42:             'name' => ['required', 'string', 'max:255'],
43:             'email' => ['required', 'string', 'lowercase', 'email', 'max:255',
'unique:'.User::class],
44:             'password' => ['required', 'confirmed', Rules\Password::defaults()],
45:         ]);
46:
```

Registration inserts User with Hash::make

File: `app/Http/Controllers/Auth/RegisteredUserController.php`. Line 50 through 54.

```
50:         $user = User::create([
51:             'name' => $request->name,
52:             'email' => $request->email,
53:             'password' => Hash::make($request->password),
54:         ]);
```

Login checks existing credentials and remember value

File: `app/Http/Requests/Auth/LoginRequest.php`. Line 56.

```
56:         if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
```

Successful login rotates the session ID

File: `app/Http/Controllers/Auth/AuthenticatedSessionController.php`. Line 39.

```
39:         $request->session()->regenerate();
```

Question 3: What is the purpose of the auth middleware?

Point to the product group, not the earlier profile group. The product group is the second auth group in web.php. Its check runs before the controller.

Say this while pointing:

"This auth middleware group requires login for all six Product routes. A guest request is redirected to the named login route before ProductController runs."

Product routes: auth applied to the whole group

File: `routes/web.php`. Line 42 through 60.

```
42: Route::middleware('auth')->group(function () {
43:     Route::get('/product', [ProductController::class, 'index'])
44:         ->name('product.index');
45:
46:     Route::get('/product/create', [ProductController::class, 'create'])
47:         ->name('product.create');
48:
49:     Route::post('/product', [ProductController::class, 'store'])
50:         ->name('product.store');
51:
52:     Route::get('/product/{product}/edit', [ProductController::class, 'edit'])
53:         ->name('product.edit');
54:
55:     Route::put('/product/{product}', [ProductController::class, 'update'])
56:         ->name('product.update');
57:
58:     Route::delete('/product/{product}', [ProductController::class, 'destroy'])
59:         ->name('product.destroy');
60: });
```

Named login destination used by the guest redirect

File: `routes/auth.php`. Line 27 through 28.

```
27:     Route::get('login', [AuthenticatedSessionController::class, 'create'])
28:         ->name('login');
```

Question 4: Why should Product CRUD routes be protected from unauthenticated users?

Show that read and write routes share the same protection. Then point to one write method to explain the database action that a guest must not reach.

Say this while pointing:

"Guests must not directly read or change these product records. The group protects store, update, and destroy as well as the list/forms. Hiding buttons alone would not prevent a direct request."

All product display/write routes protected

File: `routes/web.php`. Line 42 through 60.

```
42: Route::middleware('auth')->group(function () {
43:     Route::get('/product', [ProductController::class, 'index'])
44:         ->name('product.index');
45:
46:     Route::get('/product/create', [ProductController::class, 'create'])
47:         ->name('product.create');
48:
49:     Route::post('/product', [ProductController::class, 'store'])
50:         ->name('product.store');
51:
52:     Route::get('/product/{product}/edit', [ProductController::class, 'edit'])
53:         ->name('product.edit');
54:
55:     Route::put('/product/{product}', [ProductController::class, 'update'])
56:         ->name('product.update');
57:
58:     Route::delete('/product/{product}', [ProductController::class, 'destroy'])
59:         ->name('product.destroy');
60: });
```

Insert after validation

File: `app/Http/Controllers/ProductController.php`. Line 65.

```
65:         Product::create($data);
```

Persist changes to the selected product

File: `app/Http/Controllers/ProductController.php`. Line 100.

```
100:         $product->update($data);
```

Permanently delete the selected product

File: `app/Http/Controllers/ProductController.php`. Line 112.

```
112:         $product->delete();
```

Question 5: What is the role of SMTP configuration in password recovery?

Show the config keys read from the environment, then the broker calls for sending versus resetting. Never display the real .env password during the demonstration.

Say this while pointing:

"The SMTP config tells the mailer how to deliver the reset notification. sendResetLink asks the password broker to create/send a reset request. Later reset verifies the submitted token before the password-changing callback runs."

Default mailer selected from MAIL_MAILER

File: `config/mail.php`. Line 9.

```
9:     'default' => env('MAIL_MAILER', 'log'),
```

SMTP transport reads environment host/port/credentials

File: `config/mail.php`. Line 15 through 24.

```
15:         'smtp' => [
16:             'transport' => 'smtp',
17:             'scheme' => env('MAIL_SCHEME'),
18:             'url' => env('MAIL_URL'),
19:             'host' => env('MAIL_HOST', '127.0.0.1'),
20:             'port' => env('MAIL_PORT', 2525),
21:             'username' => env('MAIL_USERNAME'),
22:             'password' => env('MAIL_PASSWORD'),
23:             'timeout' => null,
24:             'local_domain' => env('MAIL_EHLO_DOMAIN', parse_url((string) env('APP_URL',
'http://localhost'), PHP_URL_HOST)),
```

Request reset token/email

File: `app/Http/Controllers/Auth/PasswordResetLinkController.php`. Line 41 through 43.

```
41:         $status = Password::sendResetLink(
42:             $request->only('email')
43:         );
```

Verify reset request, then callback changes password

File: `app/Http/Controllers/Auth/NewPasswordController.php`. Line 49 through 61.

```
49:         $status = Password::reset(
50:             $request->only('email', 'password', 'password_confirmation', 'token'),
51:             // This closure runs after token verification. use($request) makes the outer Request
52:             // available inside it. $user is the account selected by the broker.
53:             function (User $user) use ($request) {
54:                 // Set only the explicitly chosen password hash and a new random remember token.
55:                 // forceFill bypasses fillable, so it is limited to these server-selected keys,
56:                 // not arbitrary form input. save() persists the existing User.
57:                 $user->forceFill([
58:                     'password' => Hash::make($request->password),
59:                     'remember_token' => Str::random(60),
60:                 ])->save();
61:
```

Question 6: Why should email credentials be kept in the .env file?

Use config/mail.php to show env() reads and .gitignore to show exclusion. If the professor needs to see environment keys, use the safe .env.example or redact values; do not project the private file.

Say this while pointing:

"Mail credentials are deployment values read through env(). The real .env is ignored by Git, while the example is allowed. This keeps secrets separate from controller/source code; .env itself is not encryption."

Private SMTP username/password are read, not hard-coded

File: `config/mail.php`. Line 21 through 22.

```
21:             'username' => env('MAIL_USERNAME'),
22:             'password' => env('MAIL_PASSWORD'),
```

Root ignore rules exclude env files and allow only the example

File: `workspace root .gitignore`. Line 9 through 10.

```
9: **/.env*
10: !**/.env.example
```

Question 7: What is the difference between authentication and authorization?

Show Auth::attempt and the auth-only product group. Authorization is a missing additional decision here; do not pretend a policy file exists when it does not.

Say this while pointing:

"Auth::attempt verifies credentials, and this group requires the resulting session. Authorization would additionally decide whether this user may update or delete a particular product. We have no product owner/admin policy yet, so authenticated users share the list."

Authentication: credential check

File: `app/Http/Requests/Auth/LoginRequest.php`. Line 56.

```
56:         if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
```

Login boundary, not an administrator policy

File: `routes/web.php`. Line 42 through 43.

```
42: Route::middleware('auth')->group(function () {
43:     Route::get('/product', [ProductController::class, 'index'])
```

All products read without an owner filter

File: `app/Http/Controllers/ProductController.php`. Line 26.

```
26:         $products = Product::all();
```

Question 8: Does placing a route inside the auth middleware group automatically make a user an administrator? Explain your answer.

Show auth-only protection and the actual User model fields. The code does not assign an admin role or check product permissions. A fillable list is assignment configuration, not a permission system.

Say this while pointing:

"No. auth only checks signed-in state. This group contains no role or policy check, and registration does not assign an administrator role. Admin permissions would need separate data and server-side authorization rules."

Product group uses auth only

File: `routes/web.php`. Line 42 through 43.

```
42: Route::middleware('auth')->group(function () {
43:     Route::get('/product', [ProductController::class, 'index'])
```

User inherits authentication behavior

File: `app/Models/User.php`. Line 16 through 19.

```
16: class User extends Authenticatable
17: {
18:     /** @use HasFactory<UserFactory> */
19:     use HasFactory, Notifiable;
```

User mass-assignable fields, not admin permissions

File: `app/Models/User.php`. Line 26 through 30.

```
26:     protected $fillable = [
27:         'name',
28:         'email',
29:         'password',
30:     ];
```

Registration saves name/email/password, no role

File: `app/Http/Controllers/Auth/RegisteredUserController.php`. Line 50 through 54.

```
50:         $user = User::create([
51:             'name' => $request->name,
52:             'email' => $request->email,
53:             'password' => Hash::make($request->password),
54:         ]);
```

Common follow-up questions

These are practice questions in addition to the PDF's eight questions. Use the same file/start-line/search-anchor approach.

Follow-up question	File and start line	Explanation / expression
Where is login redirected to Products?	app/Http/Controllers/Auth/AuthenticatedSessionController.php, line 43	Login fallback is Products; intended preserves the original destination
Where is logout and the Login redirect?	app/Http/Controllers/Auth/AuthenticatedSessionController.php, line 53	Logout, clear session, rotate CSRF token, redirect
Where does the root URL choose Login or Products?	routes/web.php, line 20	Home checks auth state before selecting redirect
Where is Remember me handled by the server?	app/Http/Requests/Auth/LoginRequest.php, line 56	boolean(remember) is passed to the guard
Where is registration password hashing?	app/Http/Controllers/Auth/RegisteredUserController.php, line 53	Hash the password before account storage
Where is the reset token submitted?	resources/views/auth/reset-password.blade.php, line 19	Hidden token supplied from the reset URL
Where is the reset cooldown and expiry?	config/auth.php, line 40	60 minutes expiry; 60 seconds throttle
Where is CSRF in the Login form?	resources/views/auth/login.blade.php, line 28	Hidden form token generated by Blade
Where is CSRF in the Add Product form?	resources/views/products/create.blade.php, line 59	Product POST form protection
Where are product input rules?	app/Http/Controllers/ProductController.php, line 49	store rules; update has the same rules
Where are permitted product assignment fields?	app/Models/Product.php, line 24	Mass-assignment keys, separate from value validation

Follow-up question	File and start line	Explanation / expression
Where do product success messages disappear?	public/js/product-messages.js, line 13	Browser removes banner after 5000 milliseconds
Where are password Show/Hide controls?	public/js/password-visibility.js, line 25	Browser switches field presentation, not hashing
Where is the product Logout form?	resources/views/layouts/product.blade.php, line 87	POST plus CSRF calls Breeze logout

Prepare your own short explanation

For each PDF question, practice answering in one or two sentences, then open the first file in the index and explain the marked expression without reading the sample aloud. Follow it to a second file when necessary: route to controller, controller to model, broker to mail configuration, or form to receiving route.

For authorization/admin questions, explaining that the rule is absent is the accurate answer. Do not point to a theme button, fillable, CSRF, or an auth group as evidence of administrator permissions.